Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

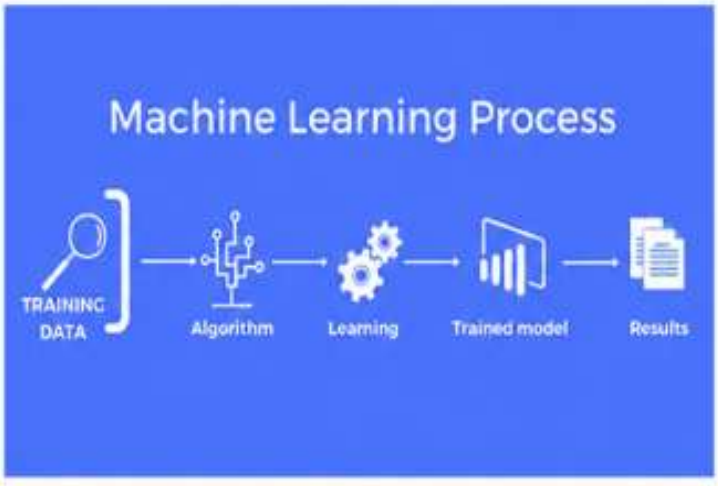
Department of Artificial Intelligence and Data Science

School of Computing

ACADEMIC YEAR 2025 - 26

Summer Semester

10211AD223 - Machine Learning Techniques Laboratory Manual



Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

NAAC A++

Recognised as CATEGORY-1 DEEMED TO BE UNIVERSITY by the UGC University Grants Commission

SCHOOL OF COMPUTING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ACADEMIC YEAR 2025 - 26

Summer Semester

10211AD223 - Machine Learning Techniques Laboratory Manual

	Prepared by	Verified by	Authorized by
Signature			
Name with TTS ID			

INDEX

S.NO.	PARTICULARS	PAGE NO
1	Syllabus	4
2	List of Experiments	7
4	**TASK 1** i. Implement the Find-S algorithm to learn the most specific hypothesis from a dataset. ii. Implement the Candidate Elimination algorithm to compute the version space for a concept. iii. Compare the outputs of Find-S and Candidate Elimination on the same dataset. Understand how these algorithms behave with noisy or incomplete data.	14
5	**TASK 2:** Perform Data Pre-processing, Data Analysis and Visualization for a given dataset. Platform: Orange, Language: Python.	19
6	**TASK 3:**Implement Principal Component Analysis (PCA) on a high-dimensional dataset. ii.Compare results of PCA with Linear Discriminant Analysis (LDA).	24
7	**TASK 4:** Use Apriori and FP-growth algorithm to find all frequent item sets for the chosen dataset and also generate Association Rules. Analyze and interpret generated rules.	26
8	**TASK 5:** Build a model using any of the classification algorithms that can effectively analyze and classify a new sample. Calculate the classification rate, accuracy, precision, recall, F1-score and confusion matrix for your data set.	36
9	**TASK 6:** Apply partitioning clustering algorithms for a given dataset Compare the results and comment on the quality of clustering using evaluation metrics.	39
10	**TASK 7:** Apply hierarchal clustering algorithms for a given dataset Compare the results and comment on the quality of clustering using evaluation metrics.	43
11	**TASK 8:** Apply back propagation neural network on image data. The idea is to build a Artificial Neural Network model that can effectively analyze and extract features from an image.	47
12	**TASK 9:**Implement a perceptron to solve a linearly separable problem. i. Demonstrate logical gate implementation using perceptron ii. Implement an MLP from scratch using Python/NumPy for a classification task. iii. Train the MLP using the backpropagation algorithm and evaluate its performance.	49
13	**TASK 10:** Hyperparameter Tuning i. Use grid search or random search to optimize hyperparameters for an MLP. Compare results before and after tuning.	53

Laboratory Experiments

Part-1

Task No	Task	CO/Skill Level
Task 1	i. Implement the Find-S algorithm to learn the most specific hypothesis from a dataset. ii. Implement the Candidate Elimination algorithm to compute the version space for a concept. iii. Compare the outputs of Find-S and Candidate Elimination on the same dataset. iv. Understand how these algorithms behave with noisy or incomplete data. **Platform: Orange, Google co-lab, Anaconda navigator, Language: Python**	CO1/S3
Task 2	i. Perform Data Pre-processing, Data Analysis and Visualization for a given dataset. Platform: Orange, Language: Python.	CO2/S3
Task 3	i. Implement Principal Component Analysis (PCA) on a high-dimensional dataset. ii. Compare results of PCA with Linear Discriminant Analysis (LDA). **Platform: Orange, Google co-lab, Anaconda navigator, Language: Python**	CO2/S3
Task 4	i. Use Apriori and FP-growth algorithm to find all frequent item sets for the chosen datasetand also generate Association Rules. ii. Analyze and interpret generated rules. **Platform: Rapidminer, Language: Python**	CO3/S3
Task 5	Build a model using any of the classification algorithms that can effectively analyze and classify a new sample. Calculate the classification rate, accuracy, precision, recall, F1-score and confusion matrix for your data set. **Platform: Orange, Google co-lab, Anaconda navigator, Language: Python**	CO3/S3
Task 6	i. Build a simple linear regression model. Evaluate the model's performance using appropriate metrics like R-squared and Mean Squared Error. Use the model to predict the target variable for a given value of the predictor. ii. Perform exploratory data analysis (EDA) to identify correlations between variables. Fit a multiple regression model using all the independent variables. Check for multicollinearity among predictors and address it if necessary. iii. Build a logistic regression model. What is the equation for the log-odds (logit function)? Interpret the coefficients of the	CO3/S3

	predictors. How do they influence the probability of the binary outcome? Evaluate the model using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Create a confusion matrix. **Platform: Google co-lab, Language: Python**	
Task 7	Apply partitioning and hierarchal clustering algorithms for a given dataset. Compare the results and comment on the quality of clustering using evaluation metrics **Platform: Google co-lab, Language: Python**	CO4/S3
Task 8	Apply back propagation neural network on image data. The idea is to build a Artificial Neural Network model that can effectively analyze and extract features from an image. **Platform: Google co-lab Language: Python**	CO5/S3
Task 9	**Perceptron** Implement a perceptron to solve a linearly separable problem. i. Demonstrate logical gate implementation using perceptron ii. Implement an MLP from scratch using Python/NumPy for a classification task. iii. Train the MLP using the backpropagation algorithm and evaluate its performance. **Platform: Google co-lab, Language: Python**	CO5/S3
Task 10	**Hyperparameter Tuning**: i. Use grid search or random search to optimize hyperparameters for an MLP. ii. Compare results before and after tuning. **Platform: Google co-lab, Language: Python**	CO5/S3

Part-2

Use Cases:

Use Case No	Use Case	CO/Skill Level
1	How can a machine learning system be developed to monitor student performance in an online education platform by incorporating **data preprocessing** (e.g., handling missing grades, normalizing time spent on activities), **classification** (e.g., predicting if a student will pass or fail a course), **clustering** (e.g., grouping students based on learning patterns or engagement levels), **perceptron** (e.g., using a simple neural network to classify students into high-risk or low-risk categories for dropout), **hyperparameter tuning** (e.g., optimizing the classification and clustering models), with **inference** (e.g., identifying underperforming students early) and **visualization** (e.g., generating dashboards showing clusters, classification probabilities, and risk levels)?	**CO1-CO5/S3**

2	How can a machine learning solution be developed for a retail store to analyze customer purchasing behavior, involving **data preprocessing** (e.g., cleaning and normalizing transaction data), **classification** (e.g., predicting whether a customer will buy a particular product), **clustering** (e.g., segmenting customers into groups based on their purchase patterns), **perceptron** (e.g., using it to build a simple binary model for predicting high-value customers), and **hyperparameter tuning** (e.g., optimizing the classification and clustering algorithms for better performance)?	**CO1-CO5/S3**
3	How can a machine learning system be designed to optimize vehicle maintenance schedules for a logistics company by using **data preprocessing** (e.g., cleaning sensor data, imputing missing values, and scaling usage metrics), **classification** (e.g., predicting whether a vehicle is likely to require maintenance soon), **clustering** (e.g., grouping vehicles based on usage patterns and wear-and-tear factors), **perceptron** (e.g., using a simple neural network to classify vehicles into "maintenance required" or "maintenance not required"), **hyperparameter tuning** (e.g., optimizing the classification and clustering algorithms), with **inference** (e.g., identifying high-risk vehicles in real time) and **visualization** (e.g., generating heatmaps and timelines of maintenance predictions and vehicle clusters)?	**CO1-CO5/S3**
4	How can a machine learning system be implemented to detect fraud in online financial transactions by incorporating **data preprocessing** (e.g., removing duplicates, handling missing values, and normalizing transaction amounts), **classification** (e.g., predicting whether a transaction is fraudulent or legitimate), **clustering** (e.g., grouping transactions to identify anomalous behavior patterns), **perceptron** (e.g., applying a basic neural network for binary fraud detection), **hyperparameter tuning** (e.g., fine-tuning the model to improve precision and recall), with **inference** (e.g., flagging suspicious transactions in real time) and **visualization** (e.g., creating dashboards to display fraud patterns and cluster distributions)?	**CO1-CO5/S3**

Total: 30 Hours
Total:60 Hours

G. Learning Resources

i. Text Books:

1. Ethem Alpaydin ,, “Introduction to Machine Learning”, 4th edition, MIT Press, 2020 (Unit 1 to 4)
2. Christopher M Bishob, “Pattern Recognition and Machine Learning”, Springer-Verlag New York Inc, 2018
3. Tom Mitchell, “Machine Learning: A Guide to Current Research”, Kluwer academic publisher, 2019.

ii. Reference Books:

1. Han, J., Pei, J., &Kamber, M. *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann., reprint, 2019

2. Oliver Theobald, “Machine Learning for Absolute Beginners", Scatterplot Press, third edition, 2020
3. Beattie, Guy, “Machine Learning: Start with Machine Learning with all new tipstricks2020.Understand how machine learning can help”, 2020

iii. Online References:

1. Machine Learning, accessed online April 20, 2021 [online] https://www.coursera.org/learn/machine-learning
2. Machine Learning by Georgia Tech, accessed online April 20, 2021[online] Available: https://www.udacity.com/course/machine-learning--ud2
3. Data mining, Stanford University, Accessed on 19th June 2021, [Online] http://statweb.stanford.edu/~lpekelis/13_datafest_cart/WekaManual-3-7-8.pdf
4. Data Warehouse: Edgewood Solutions, Accessed on 28th June 2021, [Online] https://www.mssqltips.com/sqlservertip/5690/create-a-star-schema-data-model-in-sql-server-using-the-microsoft-toolset/
5. Data mining, University of Waikato, Accessed on 19th June 2021, [Online] Available :https://waikato.github.io/weka-wiki/downloading_weka/

Laboratory Experiments

Part- 1

Task No	Task	CO/Skill Level
Task 1	i. Implement the Find-S algorithm to learn the most specific hypothesis from a dataset. ii. Implement the Candidate Elimination algorithm to compute the version space for a concept. iii. Compare the outputs of Find-S and Candidate Elimination on the same dataset. iv. Understand how these algorithms behave with noisy or incomplete data. **Platform: Orange, Google co-lab, Anaconda navigator, Language: Python**	CO1/S3
Task 2	i. Perform Data Pre-processing, Data Analysis and Visualization for a given dataset. Platform: Orange, Language: Python.	CO2/S3
Task 3	i. Implement Principal Component Analysis (PCA) on a high-dimensional dataset. ii. Compare results of PCA with Linear Discriminant Analysis (LDA). **Platform: Orange, Google co-lab, Anaconda navigator, Language: Python**	CO2/S3
Task 4	i. Use Apriori and FP-growth algorithm to find all frequent item sets for the chosen dataset and also generate Association Rules. ii. Analyze and interpret generated rules. **Platform: Rapid miner, Language: Python**	CO3/S3
Task 5	Build a model using any of the classification algorithms that can effectively analyze and classify a new sample. Calculate the classification rate, accuracy, precision, recall, F1-score and confusion matrix for your data set. **Platform: Orange, Google co-lab, Anaconda navigator, Language: Python**	CO3/S3
Task 6	i. Apply partitioning clustering algorithms for a given dataset. Compare the results and comment on the quality of clustering using evaluation metrics. Platform: Google co-lab, Language: Python	CO4/S3
Task 7	ii. Apply hierarchal clustering algorithms for a given dataset. Compare the results and comment on the quality of clustering using evaluation metrics. **Platform: Google co-lab, Language: Python**	CO4/S3
Task 8	Apply back propagation neural network on image data. The idea is to build a Artificial Neural Network model that can effectively analyze and extract features from an image. **Platform: Google co-lab Language: Python**	CO5/S3
Task 9	**Perceptron** Implement a perceptron to solve a linearly separable problem. i. Demonstrate logical gate implementation using perceptron ii. Implement an MLP from scratch using Python/NumPy for a classification task. iii. Train the MLP using the backpropagation algorithm and evaluate its performance.	CO5/S3

	Platform: Google co-lab, Language: Python	
Task 10	**Hyperparameter Tuning**: i. Use grid search or random search to optimize hyperparameters for an MLP. ii. Compare results before and after tuning. **Platform: Google co-lab, Language: Python**	CO5/S3

Part-2

Use Case No	Use Case	CO/Skill Level
1	How can a machine learning system be developed to monitor student performance in an online education platform by incorporating **data preprocessing** (e.g., handling missing grades, normalizing time spent on activities), **classification** (e.g., predicting if a student will pass or fail a course), **clustering** (e.g., grouping students based on learning patterns or engagement levels), **perceptron** (e.g., using a simple neural network to classify students into high-risk or low-risk categories for dropout), **hyperparameter tuning** (e.g., optimizing the classification and clustering models), with **inference** (e.g., identifying underperforming students early) and **visualization** (e.g., generating dashboards showing clusters, classification probabilities, and risk levels)?	**CO1-CO5/S3**
2	How can a machine learning solution be developed for a retail store to analyze customer purchasing behavior, involving **data preprocessing** (e.g., cleaning and normalizing transaction data), **classification** (e.g., predicting whether a customer will buy a particular product), **clustering** (e.g., segmenting customers into groups based on their purchase patterns), **perceptron** (e.g., using it to build a simple binary model for predicting high-value customers), and **hyperparameter tuning** (e.g., optimizing the classification and clustering algorithms for better performance)?	**CO1-CO5/S3**
3	How can a machine learning system be designed to optimize vehicle maintenance schedules for a logistics company by using **data preprocessing** (e.g., cleaning sensor data, imputing missing values, and scaling usage metrics), **classification** (e.g., predicting whether a vehicle is likely to require maintenance soon), **clustering** (e.g., grouping vehicles based on usage patterns and wear-and-tear factors), **perceptron** (e.g., using a simple neural network to classify vehicles into "maintenance required" or "maintenance not required"), **hyperparameter tuning** (e.g., optimizing the classification and clustering algorithms), with **inference** (e.g., identifying high-risk vehicles in real time) and **visualization** (e.g., generating heatmaps and timelines of maintenance predictions and vehicle clusters)?	**CO1-CO5/S3**

4	How can a machine learning system be implemented to detect fraud in online financial transactions by incorporating **data preprocessing** (e.g., removing duplicates, handling missing values, and normalizing transaction amounts), **classification** (e.g., predicting whether a transaction is fraudulent or legitimate), **clustering** (e.g., grouping transactions to identify anomalous behavior patterns), **perceptron** (e.g., applying a basic neural network for binary fraud detection), **hyperparameter tuning** (e.g., fine-tuning the model to improve precision and recall), with **inference** (e.g., flagging suspicious transactions in real time) and **visualization** (e.g., creating dashboards to display fraud patterns and cluster distributions)?	**CO1-CO5/S3**

Total: 30 Hours

CO1	**Examine the basic concepts of data mining and machine learning concepts**
Task 1:	Demonstrate the FIND-S algorithm to find a most specific hypothesis and Candidate- Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples. **Platform: Orange, Google co-lab, Anaconda navigator, Language: Python**

FIND-S Algorithm:

Objective:

Given a dataset of weather conditions with binary attributes (Sunny/Rainy, Warm/Cold, Normal/High, Strong/Weak, Warm/Cool, Same/Change), the objective is to find the most specific hypothesis based on positive examples.

Algorithm:

1.- Import necessary libraries (`matplotlib.pyplot` and `drive` from `google.colab`).
- Mount Google Drive using `drive.mount('/content/drive')`.

2.- Define a function `visualize_find_s_algorithm` that takes positive examples as input.
- Initialize the hypothesis with '0' for each attribute.
- Iterate through each positive example and update the hypothesis based on the FIND-S algorithm.
- Keep track of the visualization at each step.

3. - Define positive examples for the FIND-S algorithm.

4. - Apply the FIND-S algorithm to positive examples.
- Plot the hypothesis visualization using Matplotlib.

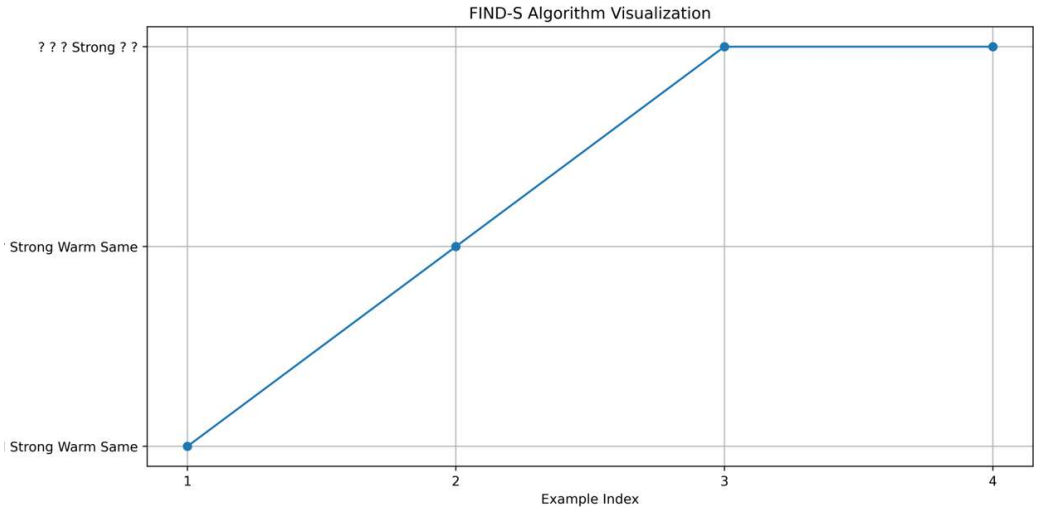
Program:

```
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
def visualize_find_s_algorithm(positive_examples):
    hypothesis = ['0'] * len(positive_examples[0])
    visualization = []
    for idx, example in enumerate(positive_examples, start=1):
        for i in range(len(example)):
            if hypothesis[i] == '0':
                hypothesis[i] = example[i]
            elif hypothesis[i] != example[i]:
                hypothesis[i] = '?'
```

```
            visualization.append((idx, " ".join(hypothesis)))
    return visualization
positive_examples_find_s = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Cool', 'Change'],
    ['Sunny', 'Hot', 'High', 'Strong', 'Cool', 'Change']
]
hypothesis_find_s_visualization = visualize_find_s_algorithm(positive_examples_find_s)
```

```
plt.figure(figsize=(12, 6))
plt.plot([point[0] for point in hypothesis_find_s_visualization], [point[1] for point in
hypothesis_find_s_visualization], marker='o')
plt.title('FIND-S Algorithm Visualization')
plt.xlabel('Example Index')
plt.ylabel('Hypothesis')
plt.xticks(range(1, len(positive_examples_find_s) + 1))
plt.grid(True)
plt.savefig('/content/drive/MyDrive/ML Lab/ExNo02/FIND-S Algorithm Visualization.png',
dpi=500)
plt.show()
```

Output:



Candidate-Elimination Algorithm:

Scenario:

Similar to the FIND-S scenario, but now we also have negative examples. The goal is to maintain a version space that includes all hypotheses consistent with positive examples and eliminates inconsistent hypotheses based on negative examples.

Algorithm:

1. Initialization:
   - Initialize the specific hypothesis with '0's.
   - Initialize the general hypothesis with '?'s.
2. Positive Examples Loop:
   - For each positive example:
     - Update the specific hypothesis:
       - If an attribute in the specific hypothesis is '0', update it with the corresponding attribute from the example.
       - If an attribute is different, update it to '?' indicating variability.
     - Update the general hypothesis:

- If an attribute in the example is different from the corresponding attribute in the specific hypothesis, update the general hypothesis to match the specific hypothesis.

3. - For each negative example:
   - Update the general hypothesis:
     - If an attribute in the example is different from the corresponding attribute in the specific hypothesis, update the general hypothesis to '?' indicating variability.
4. - Use matplotlib to visualize the evolution of the specific and general hypotheses.
   - Plot the specific hypothesis as circles ('o') and the general hypothesis as 'x' marks with a dashed line.
   - Save the plot to a file and display it.

Program:

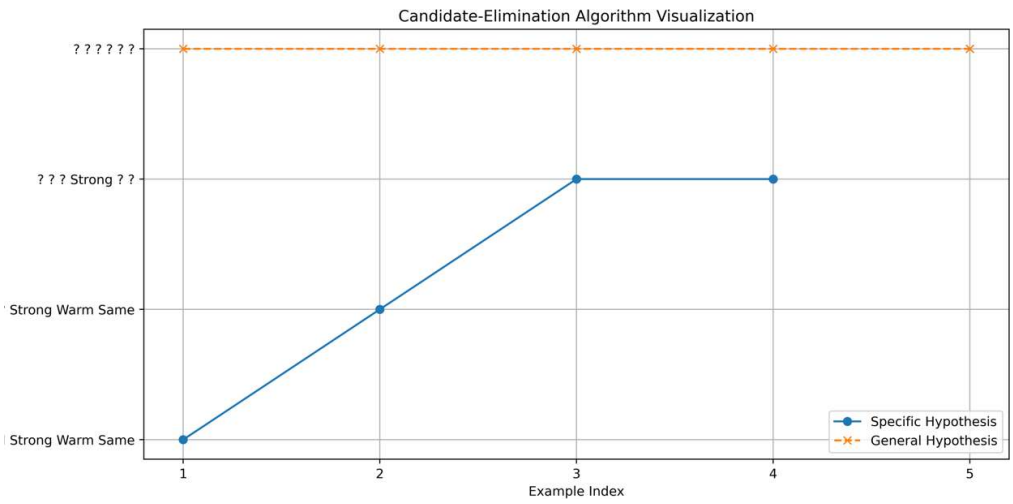
```
import matplotlib.pyplot as plt
def visualize_candidate_elimination_algorithm(positive_examples, negative_examples):
    specific_hypothesis = ['0'] * len(positive_examples[0])       general_hypothesis = ['?'] *
len(positive_examples[0])
    visualization_specific = []
    visualization_general = []
    for idx, example in enumerate(positive_examples, start=1):
        for i in range(len(example)):
            if specific_hypothesis[i] == '0':
                specific_hypothesis[i] = example[i]
            elif specific_hypothesis[i] != example[i]:
                specific_hypothesis[i] = '?'

        for i in range(len(example)):
            if example[i] != specific_hypothesis[i]:
                general_hypothesis[i] = specific_hypothesis[i]
        visualization_specific.append((idx, " ".join(specific_hypothesis)))
        visualization_general.append((idx, "  ".join(general_hypothesis)))
    for example in negative_examples:
        for i in range(len(example)):
            if example[i] != specific_hypothesis[i]:
                general_hypothesis[i] = '?'          visualization_general.append((idx + 1, "
".join(general_hypothesis)))

    return visualization_specific, visualization_general
positive_examples_candidate = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'],
    ['Rainy', 'Cold', 'Normal', 'Strong', 'Cool', 'Change'],
    ['Sunny', 'Hot', 'High', 'Strong', 'Cool', 'Change']
]
negative_examples_candidate = [
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change'],
    ['Sunny', 'Warm', 'Normal', 'Weak', 'Cool', 'Same']
]
specific, general = visualize_candidate_elimination_algorithm(positive_examples_candidate,
negative_examples_candidate)
```

```
plt.figure(figsize=(12, 6))
plt.plot([point[0] for point in specific], [point[1] for point in specific], marker='o',
label='Specific Hypothesis')
plt.plot([point[0] for point in general], [point[1] for point in general], marker='x', linestyle='--
', label='General Hypothesis')
plt.title('Candidate-Elimination Algorithm Visualization')
plt.xlabel('Example Index')
plt.ylabel('Hypothesis')
plt.legend()
plt.xticks(range(1, len(positive_examples_candidate) + 2))
plt.grid(True)
plt.savefig('/content/drive/MyDrive/ML Lab/ExNo02/Candidate-Elimination Algorithm
Visualization.png', dpi=500)
plt.show()
```

Output:



Result: Thus, the FIND-S algorithm was implemented to determine the most specific hypothesis, and the Candidate-Elimination algorithm was employed to produce a description of the set of all hypotheses consistent with the training examples successfully.

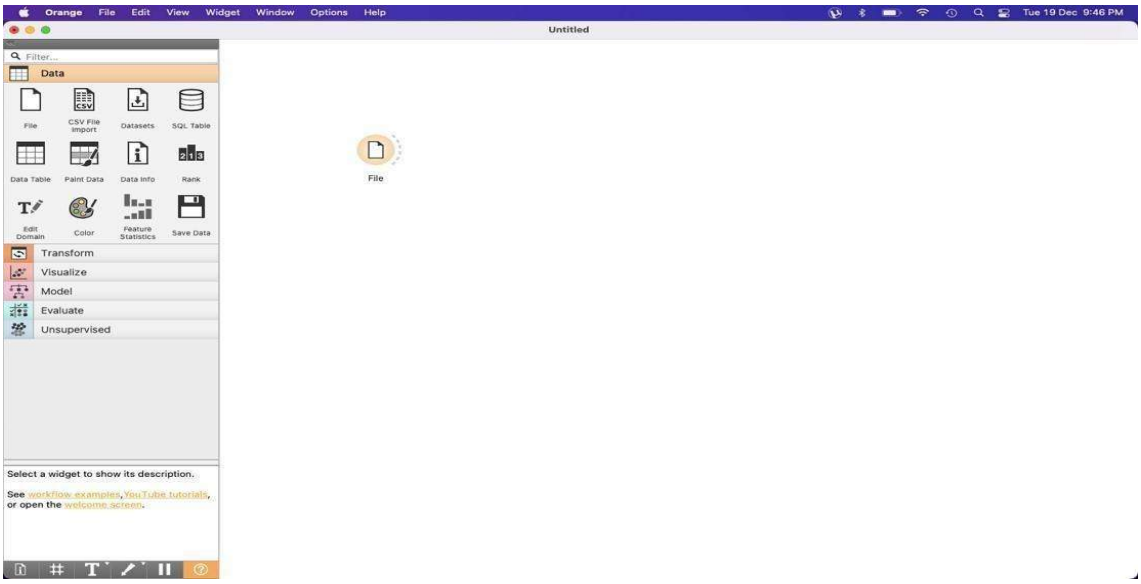
CO2	**Apply pre-processing techniques and explore the using visualization techniques**
Task 2:	Perform Data Pre-processing, Data Analysis and Visualization for a given dataset. **Platform: Orange, Language: Python**

Aim: To Perform Data Pre-processing, Data Analysis and Visualization for a given dataset.

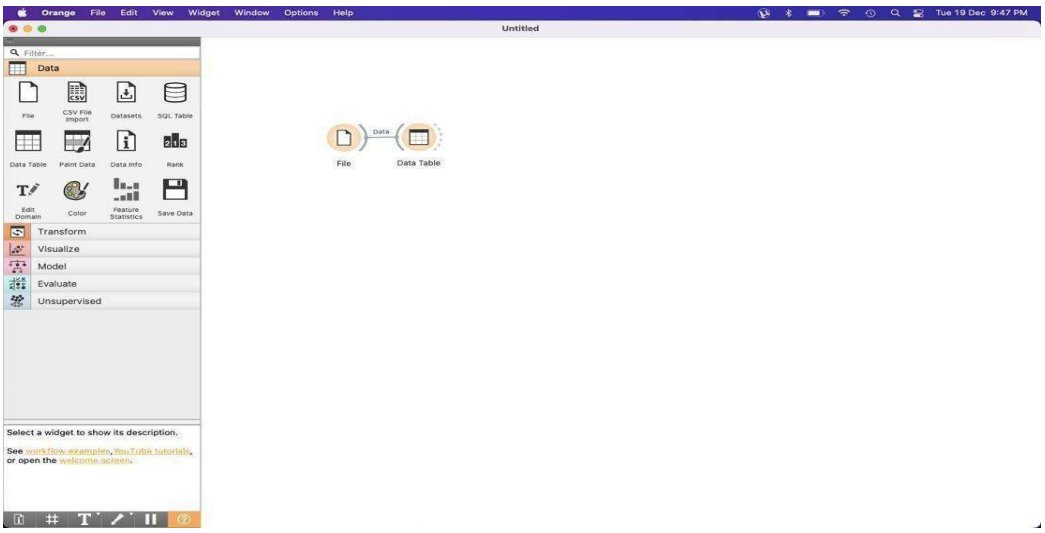
Procedure

https://orangedatamining.com/download/

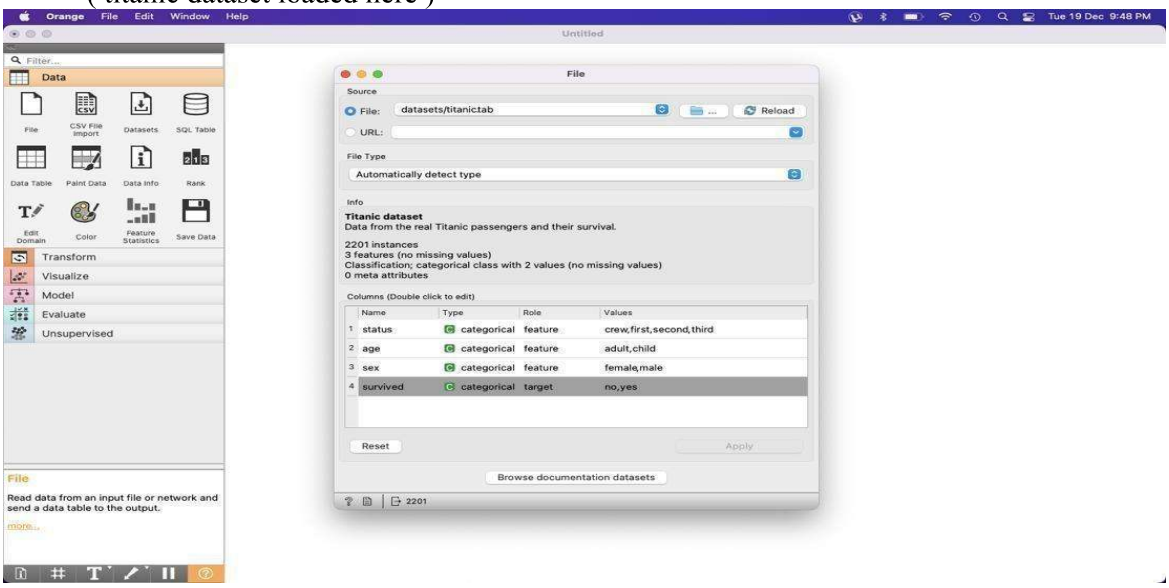
Step 1: Load the Dataset -> Drag and drop the "File" widget onto the canvas



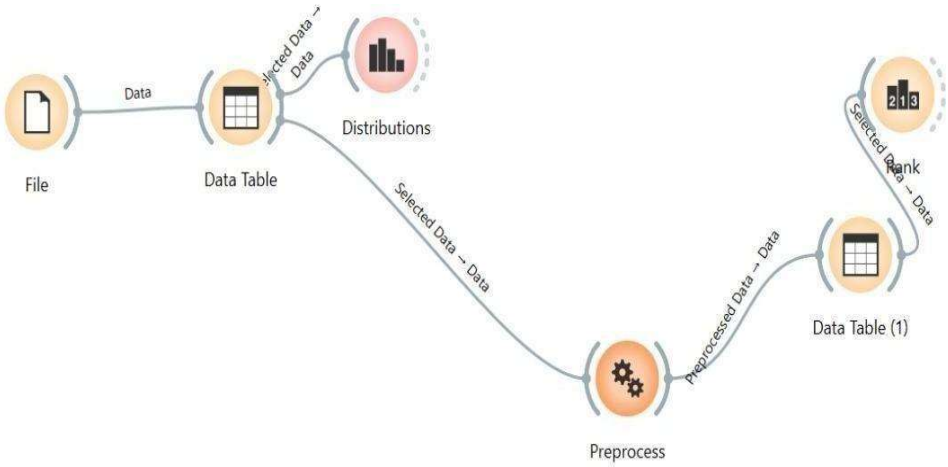
Step 2: Connect the "File" widget to the "Data Table" widget.

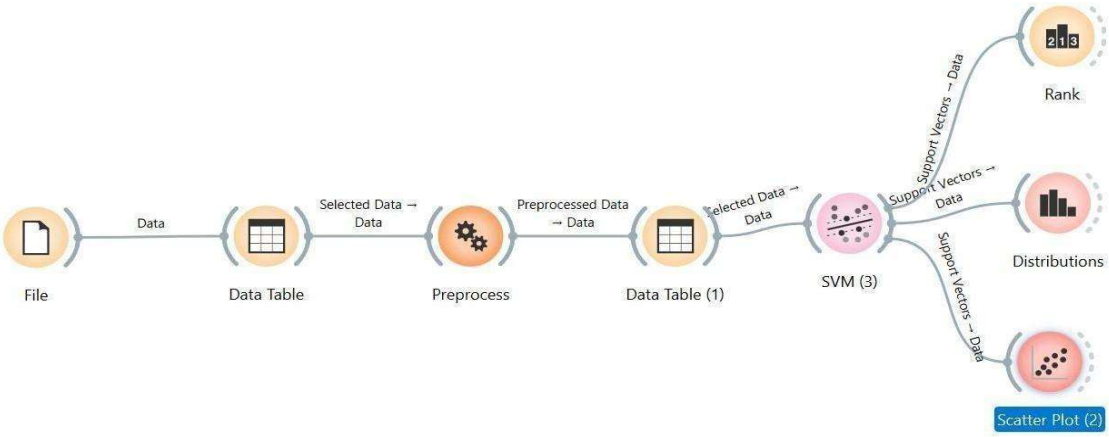


Step3: Load your customer dataset using the "Browse" button in the "File" widget.
(titanic dataset loaded here)

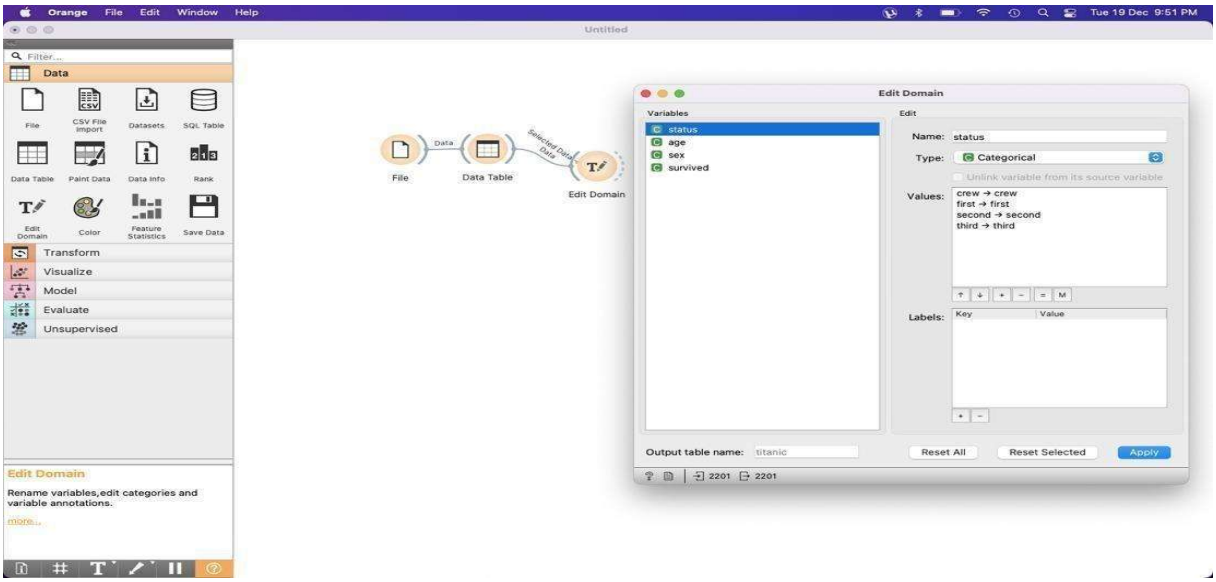


Step 4: Data Pre-processing
Drag and drop the "Data Table" widget onto the canvas.
Connect the "Data Table" widget to the "File" widget.
Use the "Data Table" widget to inspect your dataset. Address any missing values or outliers.

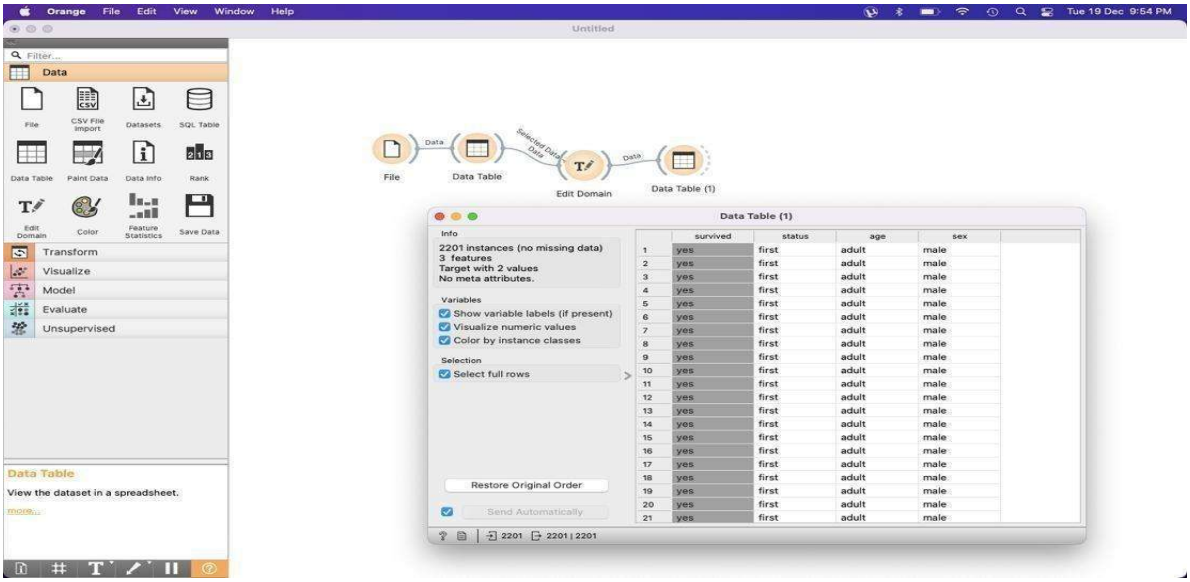




Step 5: Encode Categorical Variables
Drag and drop the "Edit Domain" widget onto the canvas.
Connect the "Edit Domain" widget to the "Data Table" widget.
In the "Edit Domain" widget, select the categorical variables and choose the appropriate encoding method.



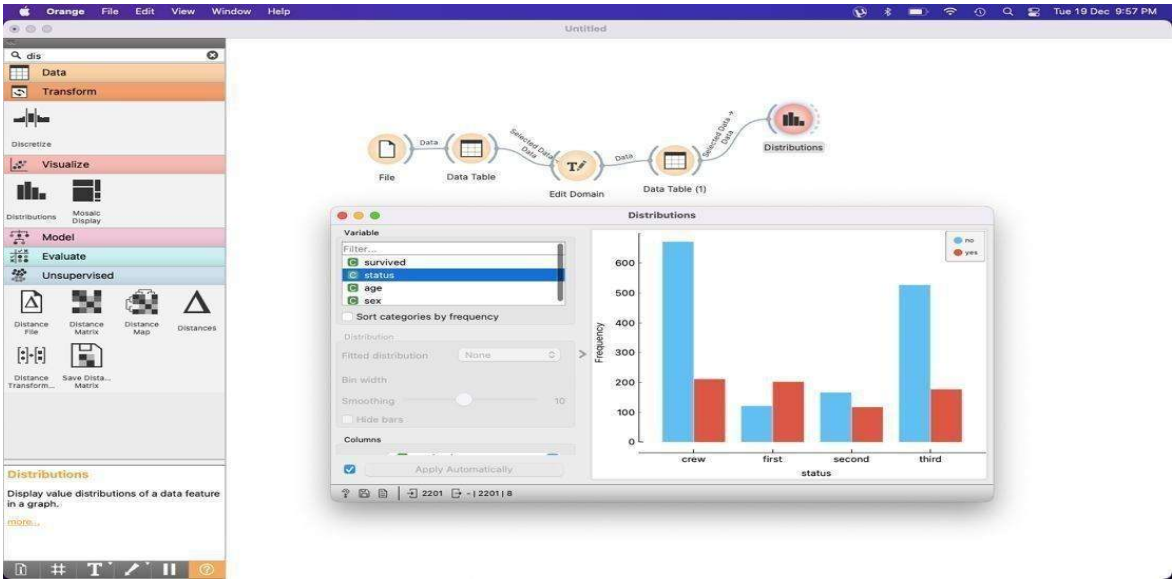
Step 6: Data Analysis
Drag and drop the "Data Table" widget onto the canvas.
Connect the "Data Table" widget to the "Edit Domain" widget.
Use the "Data Table" widget to explore basic statistics and demographics of your customers.



Step 7: Data Visualization
Drag and drop the "Distributions" widget onto the canvas.

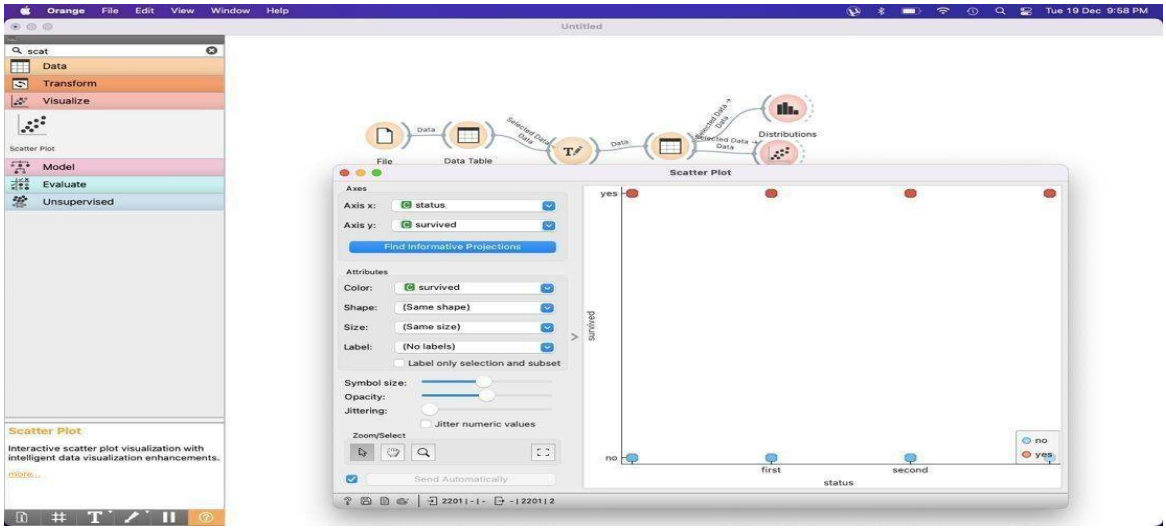
Connect the "Distributions" widget to the "Edit Domain" widget.

In the "Distributions" widget, select variables like age, income, and spending to plot histograms and visualize distributions.

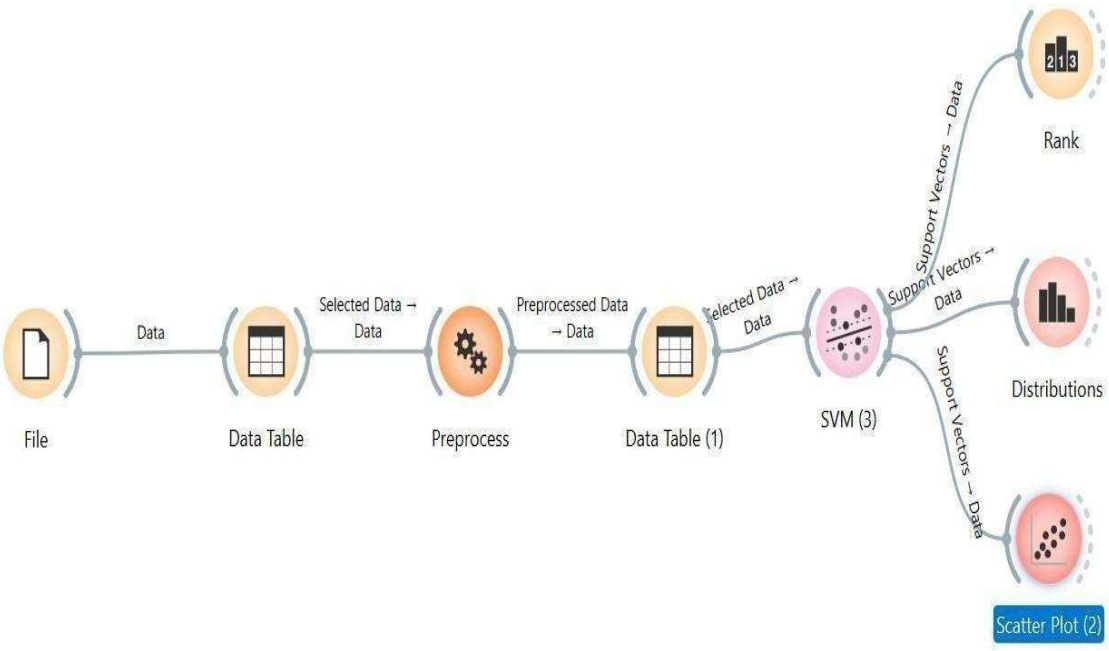


For visualizing relationships between income and spending, you can use the "Scatter Plot" widget.

Drag and drop the "Scatter Plot" widget onto the canvas.
Connect the "Scatter Plot" widget to the "Edit Domain" widget.
Choose "Income" as the X-axis variable and "Spending" as the Y-axis variable.



Final Design:



Result: Thus, data pre-processing, data analysis, and visualization for the given dataset were successfully performed using the Orange tool.

Task 3	**i. Implement Principal Component Analysis (PCA) on a high-dimensional dataset.** **ii. Compare results of PCA with Linear Discriminant Analysis (LDA).** **Platform: Orange, Google co-lab, Anaconda navigator,** **Language: Python**	**CO2/S3**

ALGORITHM:
Step1:
Generate a synthetic high-dimensional dataset: Use a multivariate normal distribution or a classification dataset like make_classification.
Step2:
Apply PCA:
Compute the principal components.
Reduce the dataset's dimensionality.
Step3:
Apply LDA:
Maximize the separation between classes while reducing dimensionality.
Step4:
Compare the results:
Use visualization (e.g., scatterplots in the 2D-reduced space).
Evaluate class separation in the reduced space using metrics like classification accuracy.
Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Step 1: Generate a high-dimensional synthetic dataset
X, y = make_classification(n_samples=500, n_features=50, n_informative=10, n_redundant=10,
                           n_classes=3, n_clusters_per_class=1, random_state=42)

# Step 2: Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 3: Apply PCA
pca = PCA(n_components=2)  # Reduce to 2 components
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# Step 4: Apply LDA
lda = LDA(n_components=2)  # Reduce to 2 components
X_train_lda = lda.fit_transform(X_train, y_train)
X_test_lda = lda.transform(X_test)

# Step 5: Visualize PCA and LDA results
plt.figure(figsize=(12, 6))
```

```
# PCA visualization
plt.subplot(1, 2, 1)
for class_value in np.unique(y_train):
    plt.scatter(X_train_pca[y_train == class_value, 0], X_train_pca[y_train == class_value, 1],
label=f"Class {class_value}")
plt.title("PCA Visualization")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()

# LDA visualization
plt.subplot(1, 2, 2)
for class_value in np.unique(y_train):
    plt.scatter(X_train_lda[y_train == class_value, 0], X_train_lda[y_train == class_value, 1],
label=f"Class {class_value}")
plt.title("LDA Visualization")
plt.xlabel("Linear Discriminant 1")
plt.ylabel("Linear Discriminant 2")
plt.legend()

plt.tight_layout()
plt.show()

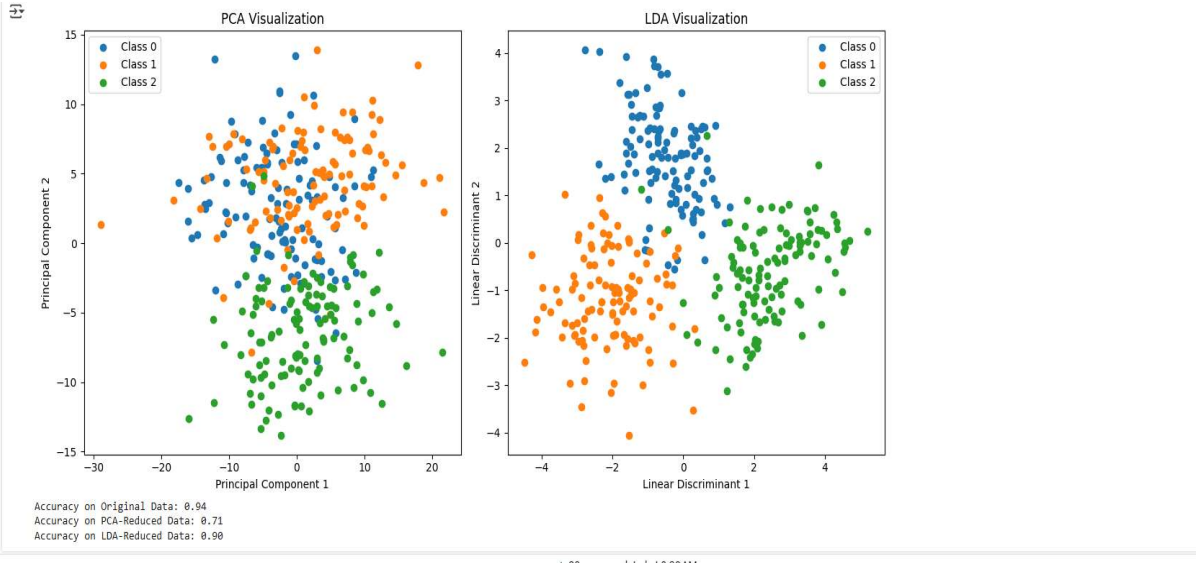
# Step 6: Compare classification performance
# Train classifier on original data
clf_original = RandomForestClassifier(random_state=42)
clf_original.fit(X_train, y_train)
y_pred_original = clf_original.predict(X_test)
original_accuracy = accuracy_score(y_test, y_pred_original)

# Train classifier on PCA-transformed data
clf_pca = RandomForestClassifier(random_state=42)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
pca_accuracy = accuracy_score(y_test, y_pred_pca)

# Train classifier on LDA-transformed data
clf_lda = RandomForestClassifier(random_state=42)
clf_lda.fit(X_train_lda, y_train)
y_pred_lda = clf_lda.predict(X_test_lda)
lda_accuracy = accuracy_score(y_test, y_pred_lda)

# Print accuracy comparison
print(f"Accuracy on Original Data: {original_accuracy:.2f}")
print(f"Accuracy on PCA-Reduced Data: {pca_accuracy:.2f}")
print(f"Accuracy on LDA-Reduced Data: {lda_accuracy:.2f}")
```

RESULTS:



PCA and LDA Comparisons

□ PCA-Reduced Data Accuracy: Lower because PCA does not account for class labels and may not retain discriminative features.

□ LDA-Reduced Data Accuracy: Close to the original data's accuracy because LDA effectively retains class-discriminative features even in reduced dimensions.+

CO3	**Apply various algorithms of Classification and Association.**
Task 4:	i. Use Apriori and FP-growth algorithm to find all frequent item sets for the chosen dataset and also generate Association Rules. ii. Analyze and interpret generated rules. **Platform: Rapidminer, Language: Python**

Tool: Rapidminer

<u>Apriori Algorithm:</u>

Use Case: Books Data

Objective:

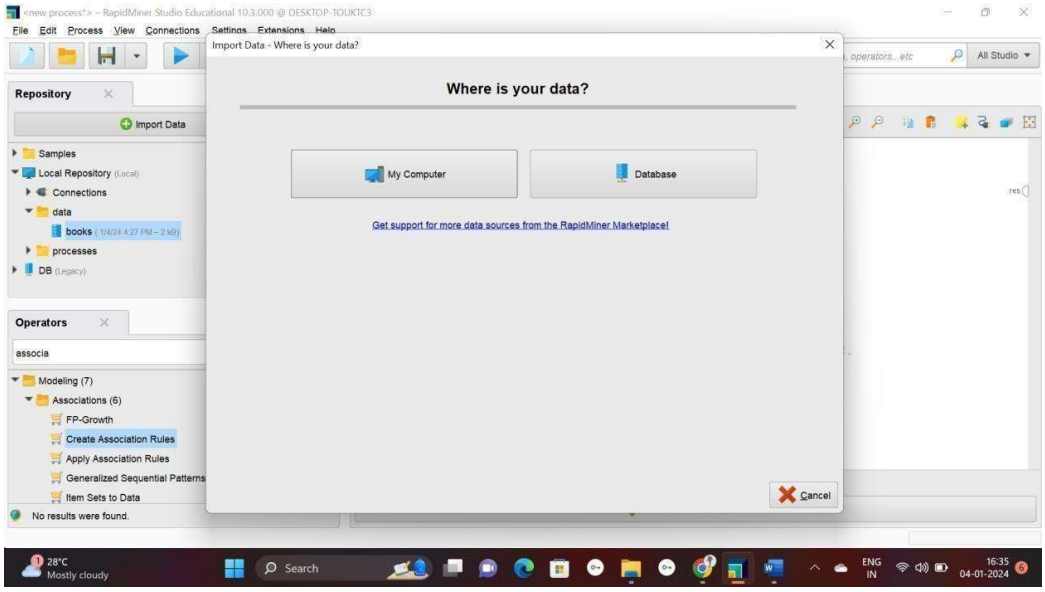
A shop wants to analyse customer purchase patterns of books to optimize product placement and run targeted promotions. The goal is to identify frequent item sets and association rules among purchased products.

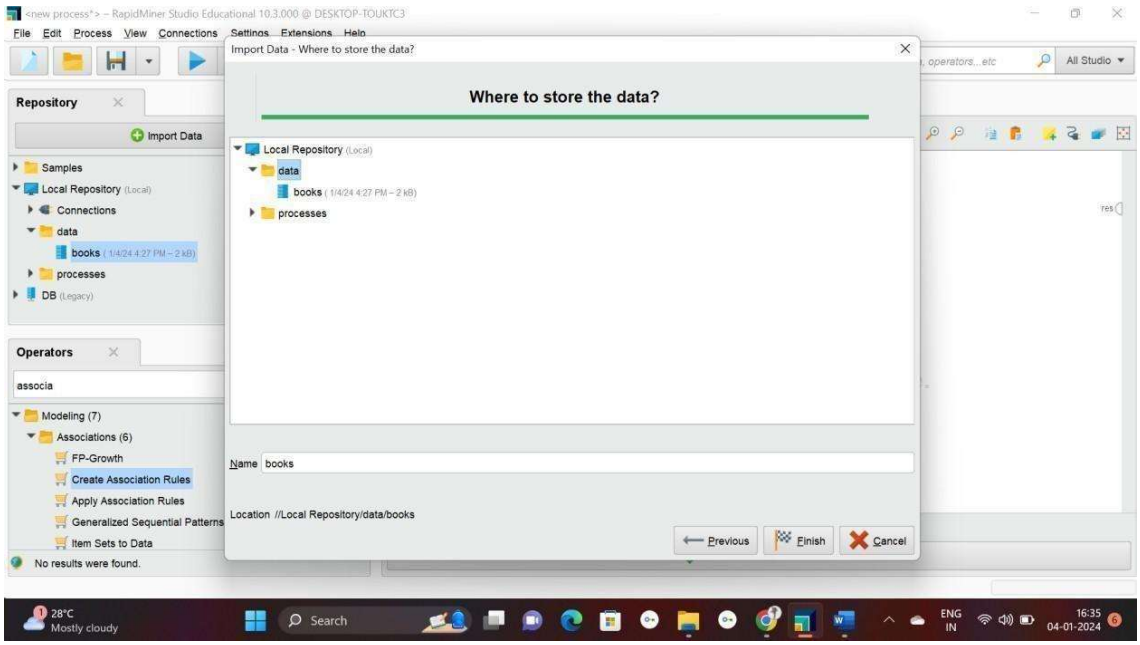
Implementation:

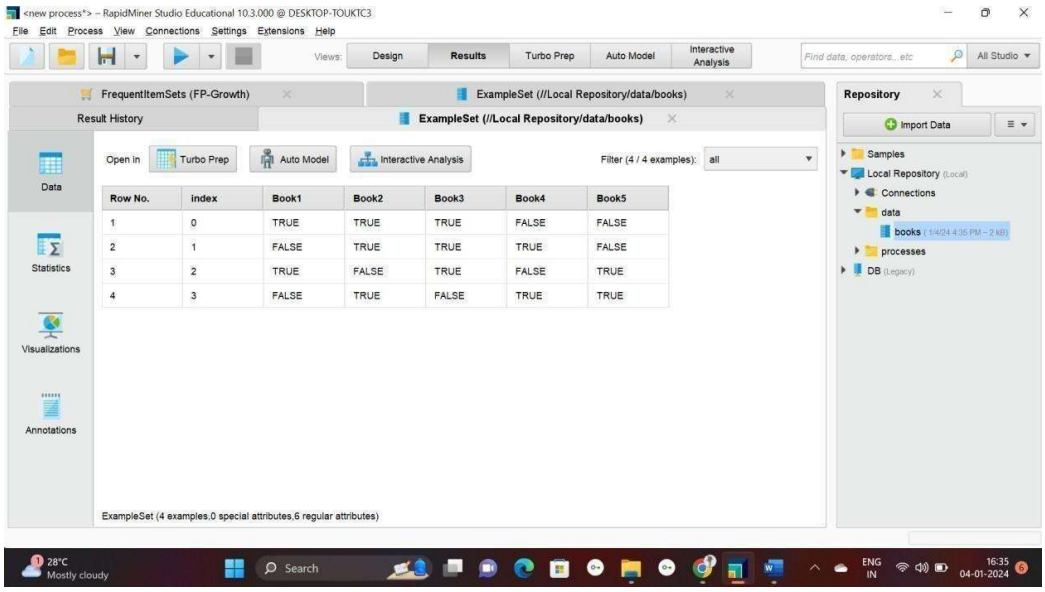
The Shop has a transaction history that includes the books purchased by customers. Using the Apriori algorithm and association rule mining, the supermarket aims to discover patterns of books that are frequently bought together.

Step 1: Load Dataset

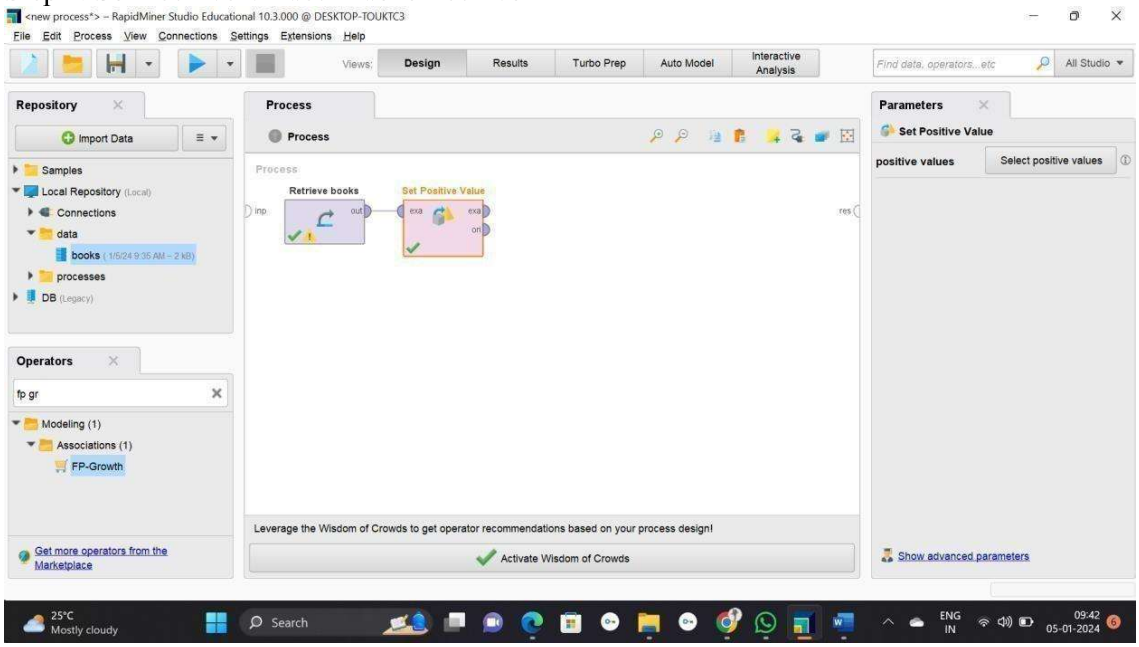
Click ->Import-> select from My computer and load, Save under Local Repository

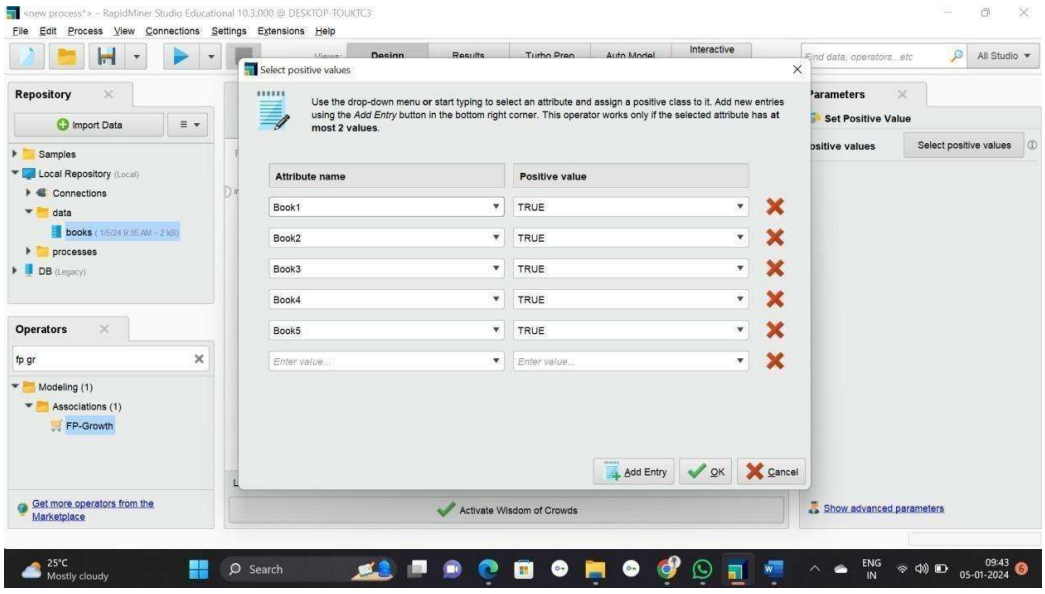

<new process*> - RapidMiner Studio Educational 10.3.000 @ DESKTOP-TOUKTC3
File Edit Process View Connections Settings Extensions Help
All Studio
Repository
Import Data
Samples
Local Repository (Local)
Connections
data
books (1/4/24 4:27 PM – 2 kB)
processes
DB (Legacy)
Operators
associa
Modeling (7)
Associations (6)
FP-Growth
Create Association Rules
Apply Association Rules
Generalized Sequential Patterns
Item Sets to Data
No results were found.
Import Data - Where is your data?
Where is your data?
My Computer
Database
Get support for more data sources from the RapidMiner Marketplace!
Cancel
28°C
Mostly cloudy
Search
ENG
IN
16:35
04-01-2024


Import Data - Where to store the data?
Where to store the data?
Local Repository (Local)
data
books (1/4/24 4:27 PM – 2 kB)
processes
Name books
Location //Local Repository/data/books
Previous
Finish
Cancel
Repository
Import Data
Samples
Local Repository (Local)
Connections
data
books (1/4/24 4:27 PM – 2 kB)
processes
DB (Legacy)
Operators
associa
Modeling (7)
Associations (6)
FP-Growth
Create Association Rules
Apply Association Rules
Generalized Sequential Patterns
Item Sets to Data
No results were found.

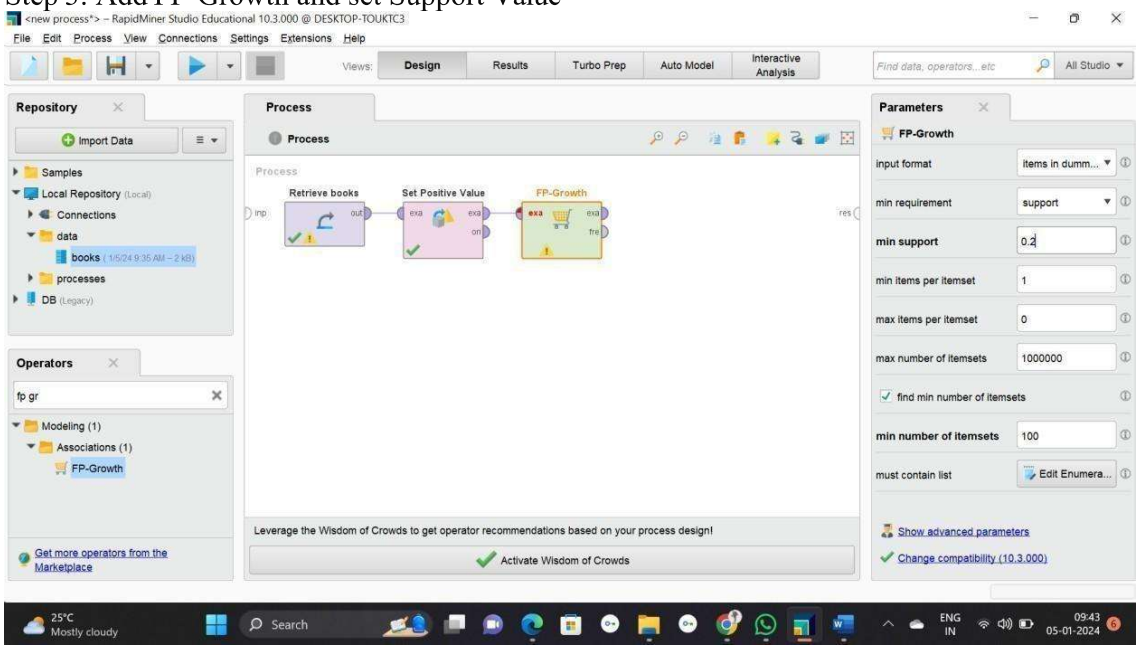

FrequentItemSets (FP-Growth)
ExampleSet (//Local Repository/data/books)
Result History
ExampleSet (//Local Repository/data/books)
Open in
Turbo Prep
Auto Model
Interactive Analysis
Filter (4 / 4 examples):
all
Row No. | index | Book1 | Book2 | Book3 | Book4 | Book5
1 | 0 | TRUE | TRUE | TRUE | FALSE | FALSE
2 | 1 | FALSE | TRUE | TRUE | TRUE | FALSE
3 | 2 | TRUE | FALSE | TRUE | FALSE | TRUE
4 | 3 | FALSE | TRUE | FALSE | TRUE | TRUE
Data
Statistics
Visualizations
Annotations
ExampleSet (4 examples,0 special attributes,6 regular attributes)
Repository
Import Data
Samples
Local Repository (Local)
Connections
data
books (1/4/24 4:35 PM – 2 kB)
processes
DB (Legacy)

Step 2: Set Positive Values True to Positive

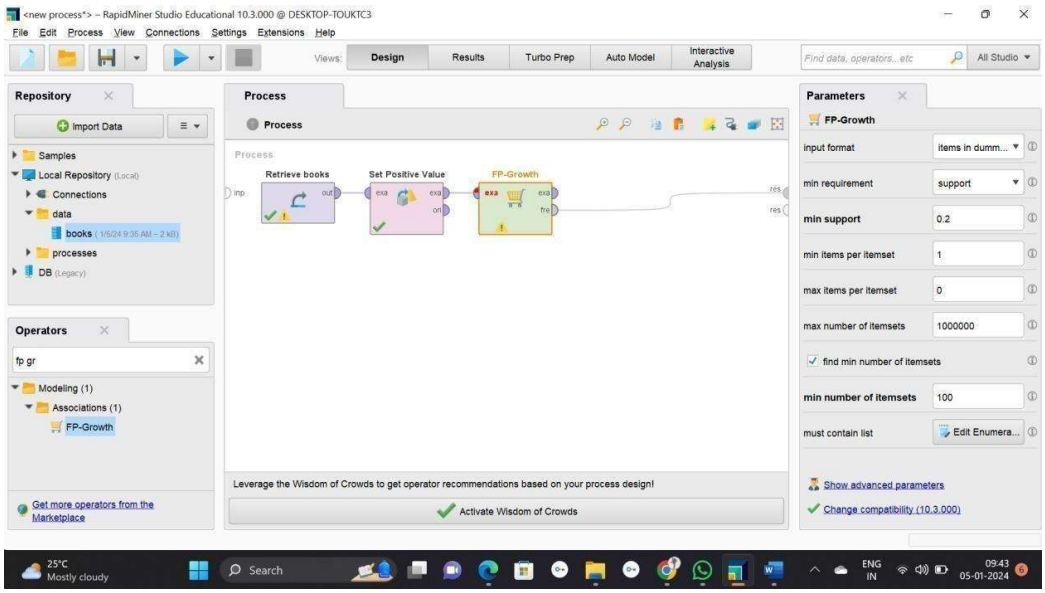


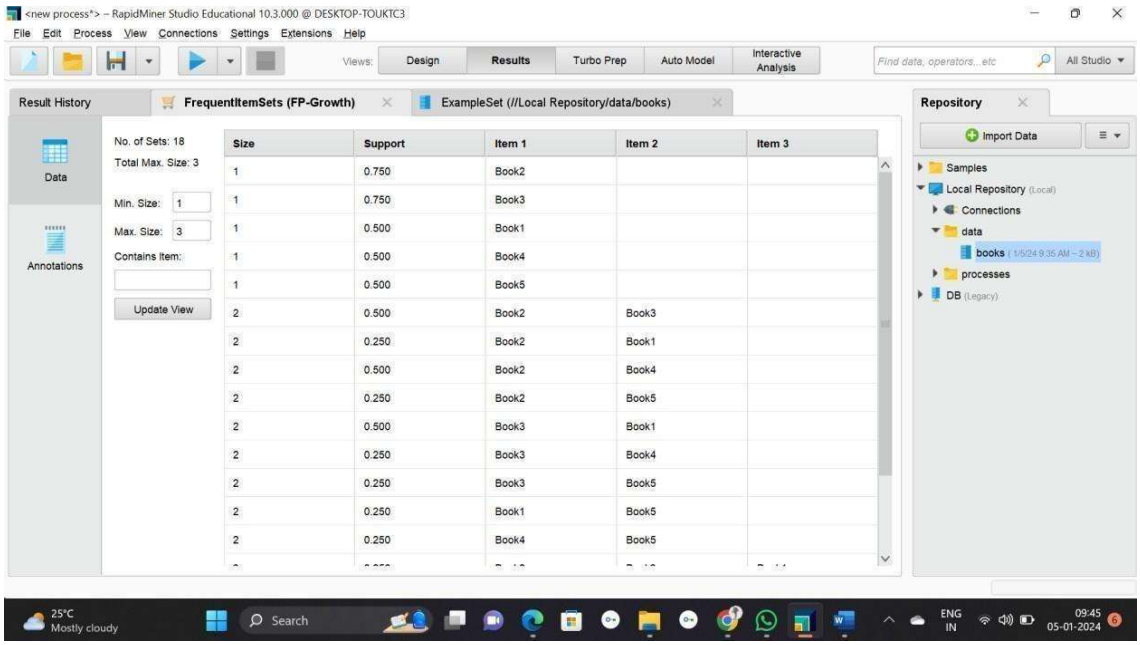


Step 3: Add FP-Growth and set Support Value

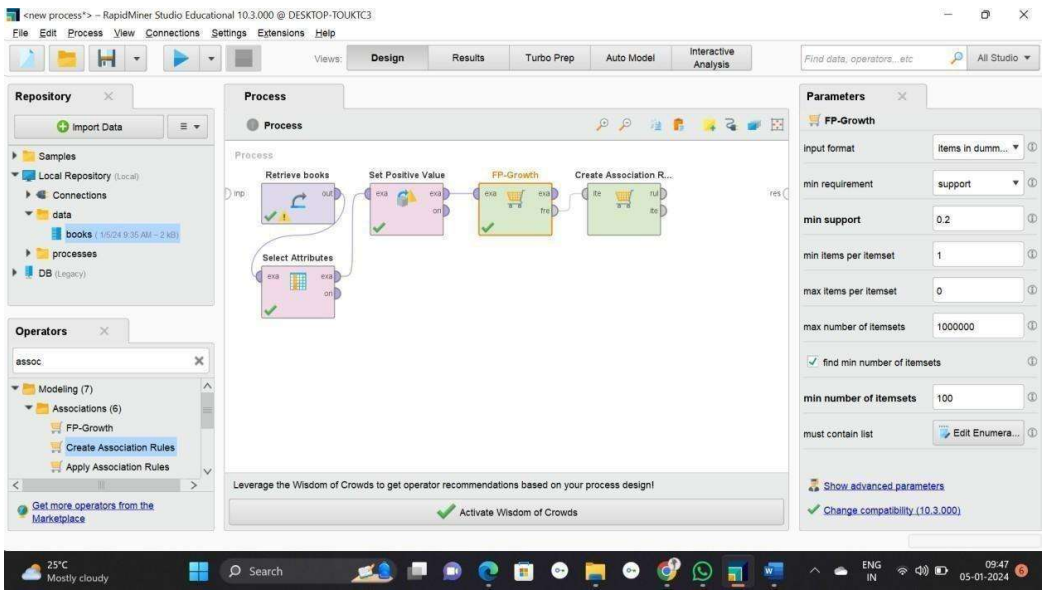


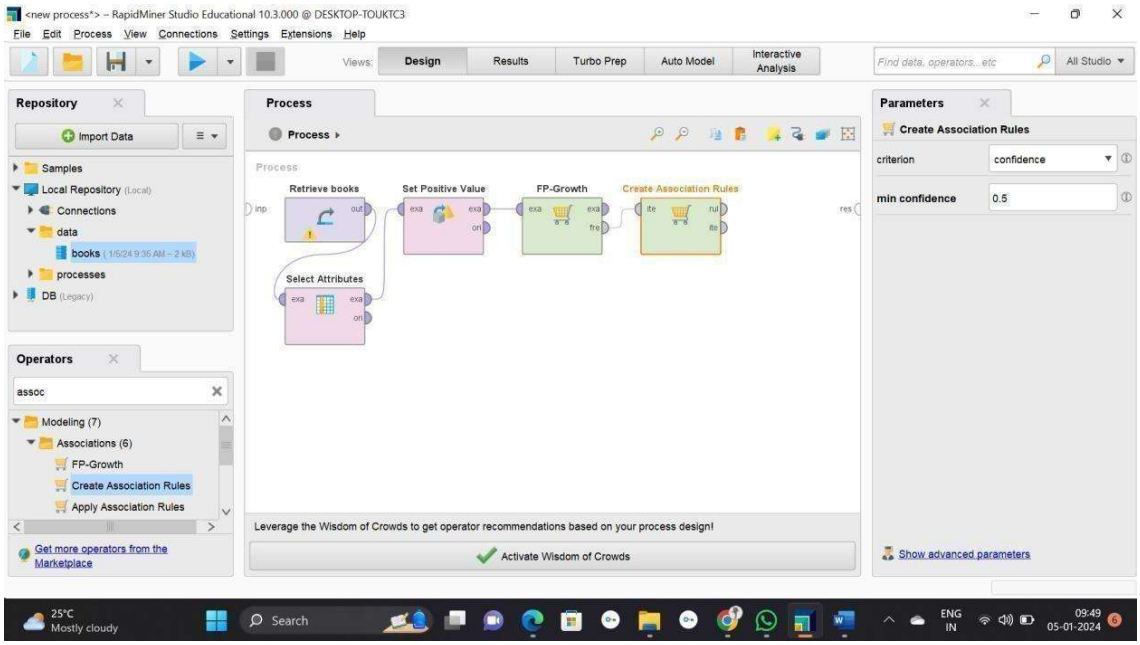
Step 4: View the Frequent Itemset



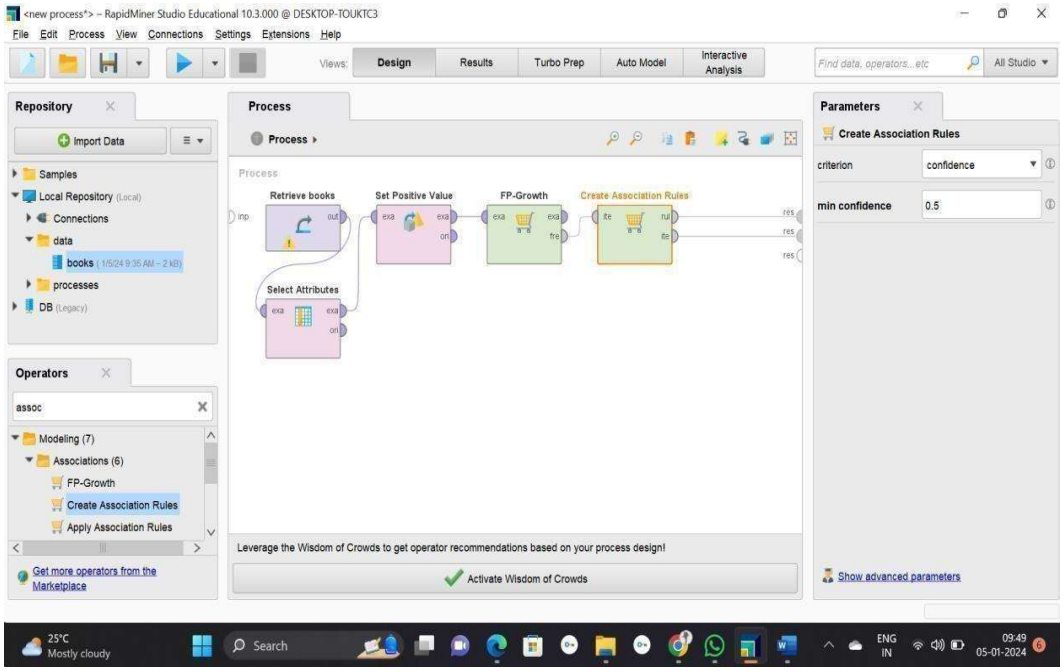


Step 5: Add Create Association and set the confidence value

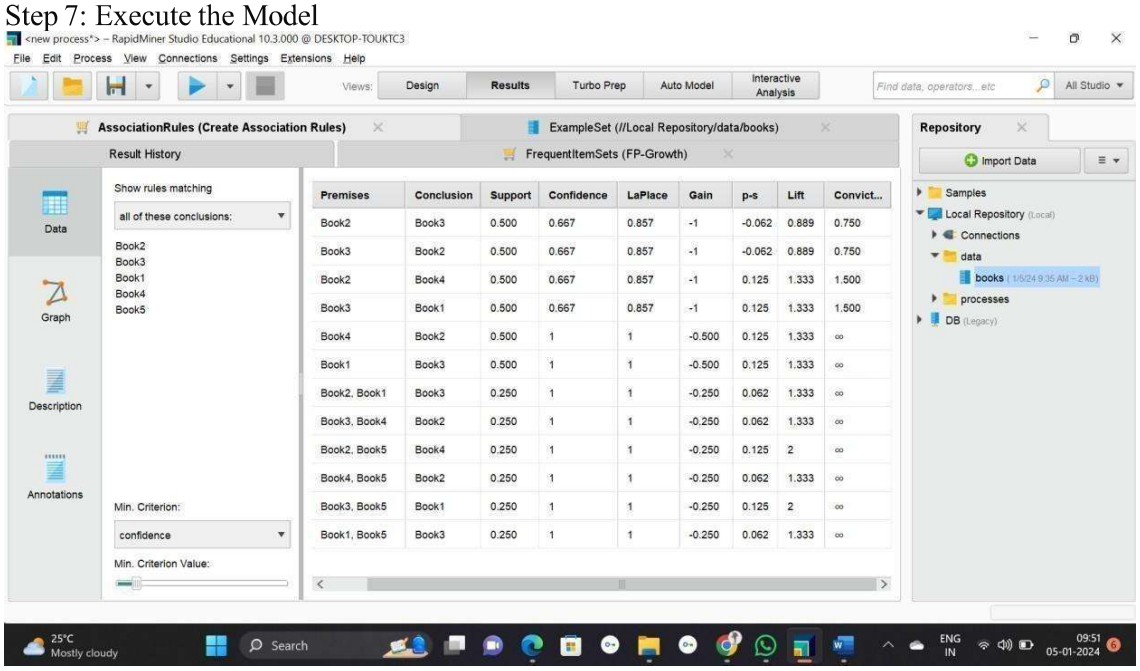




Step 6: View both rules and Item set



Step 7: Execute the Model



Apriori Algorithm: Tool : Google Colab Use Case: Books Data Objective:

The main objective is to identify sets of books that frequently occur together in the dataset. These sets are known as frequent itemsets, from the frequent itemsets, generate association rules that express relationships between different sets of books and Visualize the generated association rules as a directed graph.

Implementation:

Using the Apriori algorithm and association rule mining, the prediction is brought to discover patterns that are frequently happens together.

Program:

```
# Install necessary libraries
!pip install mlxtend import
pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
# Sample dataset (replace this with your actual dataset) buying_books_data =
[
    ['Book1', 'Book2', 'Book3'],
    ['Book2', 'Book3', 'Book4'],
    ['Book1', 'Book3', 'Book5'],
    ['Book2', 'Book4', 'Book5'],
]
# Convert the dataset to a one-hot encoded format te =
TransactionEncoder()
te_ary = te.fit(buying_books_data).transform(buying_books_data) df_buying_books =
pd.DataFrame(te_ary, columns=te.columns_)
```

```
# Apply the Apriori algorithm min_support = 0.2  #
Adjust as needed
frequent_itemsets = apriori(df_buying_books, min_support=min_support, use_colnames=True)
# Generate association rules min_confidence = 0.5
# Adjust as needed
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=min_confidence)
# Display frequent itemsets
print("Frequent Itemsets:") print(frequent_itemsets)
# Display association rules print("\nAssociation
Rules:") print(rules)
```

OUTPUT:

```
Frequent Itemsets:
    support                itemsets
0      0.50                  (Book1)
1      0.75                  (Book2)
2      0.75                  (Book3)
3      0.50                  (Book4)
4      0.50                  (Book5)
5      0.25           (Bo  k2, Book1)
6      0.50           (Bo  k3, Book1)
7      0.25           (Bo  k5, Book1)
8      0.50           (Bo  k3, Book2)
9      0.50           (Bo  k4, Book2)
10     0.25           (Bo  k5, Book2)
11     0.25           (Bo  k4, Book3)
12     0.25           (Bo  k5, Book3)
13     0.25           (Bo  k5, Book4)
14     0.25   (Book3, Bo  k2, Book1)
15     0.25   (Book5, Bo  k3, Book1)
16     0.25   (Book4, Bo  k3, Book2)
17     0.25   (Book4, Bo  k5, Book2)
```

```
Association Rules:
       antecedents     consequents antecedent support consequent
support \
0           (Book1)          (Book2)                 0.50
0.75
1           (Book3)          (Book1)                 0.75
0.50
2           (Book1)          (Book3)                 0.50
0.75
3           (Book5)          (Book1)                 0.50
0.50
4           (Book1)          (Book5)                 0.50
0.50
            (Book3)          (Book2)                 0.75
0.75
            (Book2)          (Book3)                 0.75
0.75
            (Book4)          (Book2)                 0.50
0.75
            (Book2)          (Book4)                 0.75
0.50
            (Book5)          (Book2)                 0.50
0.75
            (Book4)          (Book3)                 0.50
```

```
11          (Book5)          (Book3)                  0.50
0.75
12          (Book5)          (Book4)                  0.50
0.50
13          (Book4)          (Book5)                  0.50
0.50
14   (Book2, Book3)          (Book1)                  0.50
0.50
15   (Book3, Book1)          (Book2)                  0.50
0.75
16   (Book2, Book1)          (Book3)                  0.25
0.75
17          (Book1)   (Book2, Book3)                  0.50
0.50
18   (Book5, Book3)          (Book1)                  0.25
0.50
19   (Book5, Book1)          (Book3)                  0.25
0.75
20   (Book3, Book1)          (Book5)                  0.50
0.50
21          (Book5)   (Book3, Book1)                  0.50
0.50
22          (Book1)   (Book5, Book3)                  0.50
0.25
23  (Book3, Book4)          (Book2)                  0.25
0.75
24   (Book2, Book4)          (Book3)                  0.50
0.75
25   (Book2, Book3)          (Book4)                  0.50
0.50
26          (Book4)   (Book2, Book3)                  0.50
0.50
27   (Book5, Book4)          (Book2)                  0.25
0.75
28   (Book2, Book4)          (Book5)                  0.50
0.50
29   (Book5, Book2)          (Book4)                  0.25
0.50
30          (Book4)   (Book5, Book2)                  0.50
0.25
31          (Book5)   (Book2, Book4)                  0.50
0.50
```

	support	confidence	lift	leverage	conviction	zhangs_metric
0	0.25	0.500000	0.666667	-0.1250	0.50	-0.500000
1	0.50	0.666667	1.333333	0.1250	1.50	1.000000
2	0.50	1.000000	1.333333	0.1250	inf	0.500000
3	0.25	0.500000	1.000000	0.0000	1.00	0.000000
4	0.25	0.500000	1.000000	0.0000	1.00	0.000000
5	0.50	0.666667	0.888889	-0.0625	0.75	-0.333333
6	0.50	0.666667	0.888889	-0.0625	0.75	-0.333333
7	0.50	1.000000	1.333333	0.1250	inf	0.500000
8	0.50	0.666667	1.333333	0.1250	1.50	1.000000
9	0.25	0.500000	0.666667	-0.1250	0.50	-0.500000
10	0.25	0.500000	0.666667	-0.1250	0.50	-0.500000
11	0.25	0.500000	0.666667	-0.1250	0.50	-0.500000
12	0.25	0.500000	1.000000	0.0000	1.00	0.000000
13	0.25	0.500000	1.000000	0.0000	1.00	0.000000

14	0.25	0.500000	1.000000	0.0000	1.00	0.000000
15	0.25	0.500000	0.666667	-0.1250	0.50	-0.500000
16	0.25	1.000000	1.333333	0.0625	inf	0.333333
17	0.25	0.500000	1.000000	0.0000	1.00	0.000000
18	0.25	1.000000	2.000000	0.1250	inf	0.666667
19	0.25	1.000000	1.333333	0.0625	inf	0.333333
20	0.25	0.500000	1.000000	0.0000	1.00	0.000000
21	0.25	0.500000	1.000000	0.0000	1.00	0.000000
22	0.25	0.500000	2.000000	0.1250	1.50	1.000000
23	0.25	1.000000	1.333333	0.0625	inf	0.333333
24	0.25	0.500000	0.666667	-0.1250	0.50	-0.500000
25	0.25	0.500000	1.000000	0.0000	1.00	0.000000
26	0.25	0.500000	1.000000	0.0000	1.00	0.000000
27	0.25	1.000000	1.333333	0.0625	inf	0.333333
28	0.25	0.500000	1.000000	0.0000	1.00	0.000000
29	0.25	1.000000	2.000000	0.1250	inf	0.666667
30	0.25	0.500000	2.000000	0.1250	1.50	1.000000
31	0.25	0.500000	1.000000	0.0000	1.00	0.000000

FP-growth algorithm

Tool : Google Colab

Program:

!pip install pyfpgrowth import pyfpgrowth
Sample dataset (replace this with your actual dataset) buying_books_data = [
['Book1', 'Book2', 'Book3'],
['Book2', 'Book3', 'Book4'],
['Book1', 'Book3', 'Book5'],
['Book2', 'Book4', 'Book5'],
]

Convert the dataset to a list of transactions transactions = [tuple(transaction) for transaction in buying_books_data]

Apply the FP-growth algorithm min_support = 2 # Adjust as needed
patterns = pyfpgrowth.find_frequent_patterns(transactions, min_support)

Generate association rules min_confidence = 0.5 # Adjust as needed
rules = pyfpgrowth.generate_association_rules(patterns, min_confidence)

Display frequent itemsets print("Frequent Itemsets:") print(patterns)
Display association rules print("\nAssociation Rules:") print(rules)

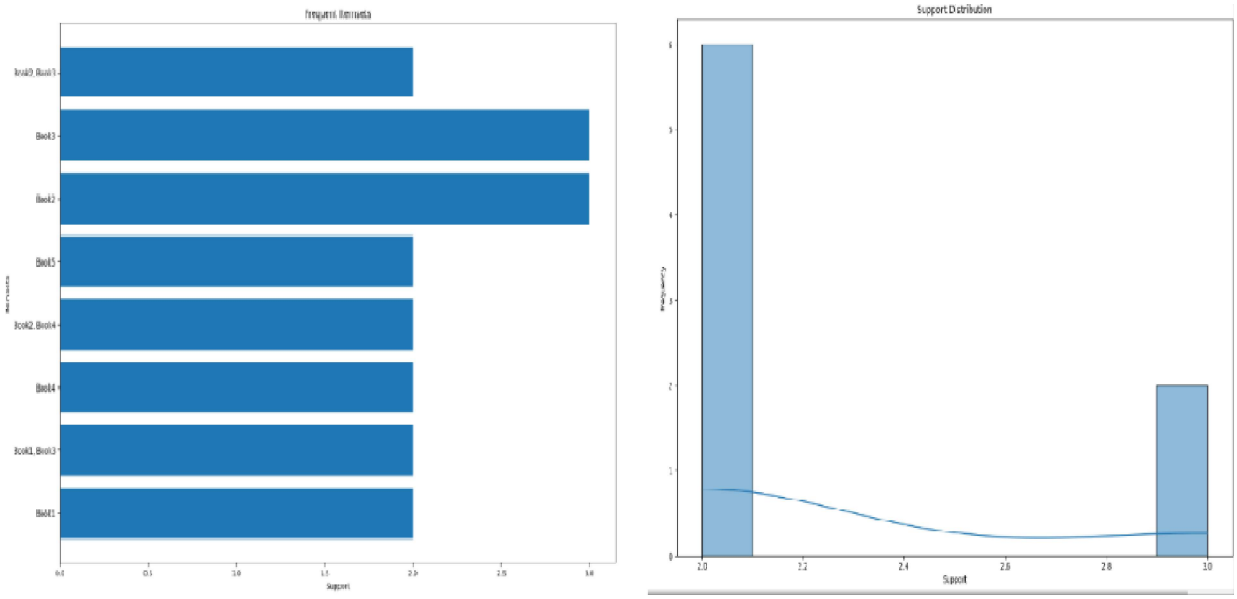
```
itemset_labels = [', '.join(map(str, itemset)) for itemset in
patterns.keys()]
```

plt.figure(figsize=(36, 24))
plt.subplot(2, 2, 1)
plt.barh(itemset_labels, list(patterns.values())) plt.xlabel('Support')

plt.ylabel('Itemsets') plt.title('Frequent Itemsets')
import seaborn as sns plt.subplot(2, 2, 2)
sns.histplot(list(patterns.values()), bins=10, kde=True) plt.xlabel('Support')
plt.ylabel('Frequency') plt.title('Support Distribution')

OUTPUT:

Frequent Itemsets:
{('Book1',): 2, ('Book1', 'Book3'): 2, ('Book4',): 2, ('Book2',
'Book4'): 2, ('Book5',): 2, ('Book2',): 3, ('Book3',): 3, ('Book2',
'Book3'): 2}
Association Rules:
{('Book1',): (('Book3',), 1.0), ('Book3',): (('Book2',), 0.6666666666666666), ('Book2',):
(('Book3',), 0.6666666666666666),
('Book4',): (('Book2',), 1.0)}



Result:

Thus, the Apriori and FP-Growth algorithms were applied to identify all frequent item sets for the chosen dataset and to generate association rules successfully.

Task 5	Build a model using any of the classification algorithms that can effectively analyze and classify a new sample. Calculate the classification rate, accuracy, precision, recall, F1-score and confusion matrix for your data set

Description:

implemented a classification model using the Random Forest algorithm, which is a powerful ensemble learning method. The dataset used is the Iris dataset, a well-known dataset in machine learning for classification tasks.

Aim:

To build a model using any of the classification algorithms that can effectively analyze and classify a new sample. Calculate the classification rate, accuracy, precision, recall, F1-score and confusion matrix for your data set.

Procedure:

Load the PDF file

- Read the input PDF file using a PDF processing library (e.g., PyPDF2, pdfplumber in Python).

Extract Text & Images

- Iterate through each page of the PDF.
- Extract text and images separately for structured conversion.

Format Text & Images

- Preserve paragraphs, line breaks, and basic formatting.
- Convert images to an appropriate format (e.g., embedded or linked in the Word document).

Create a Word Document

- Use a Word-processing library (e.g., python-docx in Python) to create a .docx file.
- Insert extracted text and images while maintaining their original layout.

Save the Word Document

- Write the formatted content to a .docx file and store it in the desired location.

Code:

```
import numpy as np
from sklearn import metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

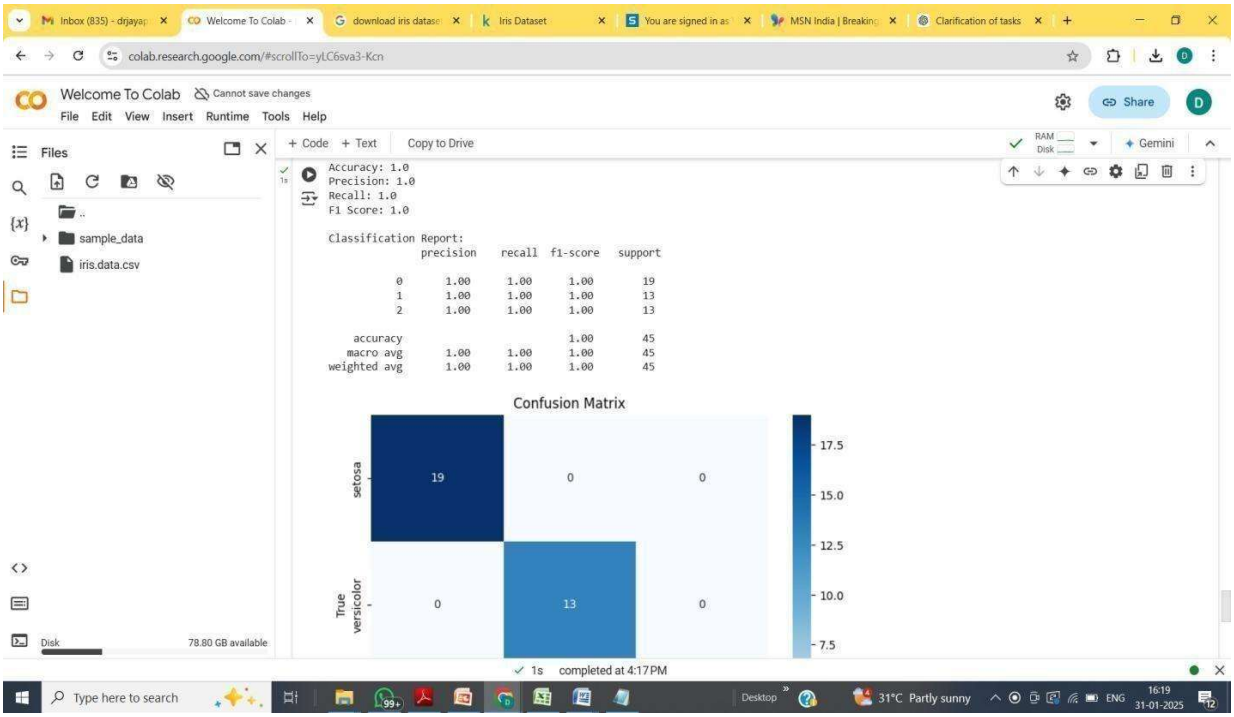
# Load the Iris dataset
iris = load_iris()

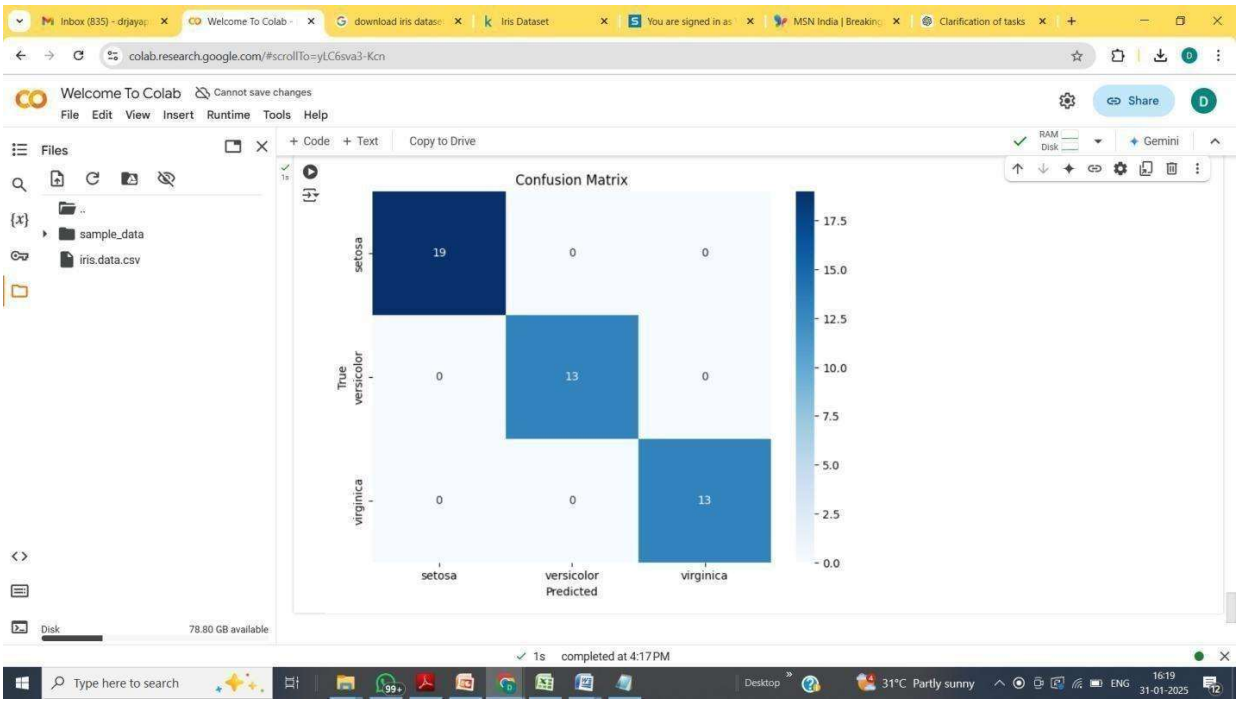
# Features (X) and target (y)
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Initialize a classifier (Random Forest in this case)
classifier = RandomForestClassifier(n_estimators=100, random_state=42)
# Train the classifier on the training data
classifier.fit(X_train, y_train)
# Make predictions on the test set
predictions = classifier.predict(X_test)
# Evaluate model performance
print("Accuracy:", accuracy_score(y_test,  predictions))
print("Precision:", precision_score(y_test, predictions, average='weighted'))
print("Recall:", recall_score(y_test, predictions, average='weighted'))
print("F1 Score:", f1_score(y_test, predictions, average='weighted'))
# Print detailed classification report
print("\nClassification Report:\n", classification_report(y_test, predictions))
# Confusion matrix
conf_matrix = metrics.confusion_matrix(y_test, predictions)
# Plot the confusion matrix using Seaborn heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=iris.target_names,
yticklabels=iris.target_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

Output





RESULT:

Thus, the model employing classification algorithms to effectively analyze and classify new samples, as well as to calculate the classification rate, accuracy, precision, recall, F1-score, and confusion matrix for the given dataset, was successfully implemented.

Task 6	**Build a simple linear regression model.** **I. Evaluate the model's performance using appropriate metrics like R-squared and Mean Squared Error. Use the model to predict the target variable for a given value of the predictor.** **ii. Perform exploratory data analysis (EDA) to identify correlations between variables. Fit a multiple regression model using all the independent variables. Check for multicollinearity among predictors and address it if necessary.** **iii. Build a logistic regression model. What is the equation for the log odds (logit function)? Interpret the coefficients of the predictors.** **How do they influence the probability of the binary outcome? Evaluate the model using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Create a confusion matrix.**

Description:

1. **Simple Linear Regression (Building the model, evaluating R-squared and MSE, making predictions)**

2. **Exploratory Data Analysis (EDA) & Multiple Regression (Correlation analysis, handling multicollinearity)**
3. **Logistic Regression (Fitting the model, interpreting coefficients, evaluating with classification metrics, and generating a confusion matrix)**

The program uses pandas, numpy, seaborn, matplotlib, sklearn for machine learning tasks, and statsmodels for statistical summaries.

Step 1: Load and Preprocess the Dataset

Import necessary libraries (pandas, numpy, seaborn, matplotlib, sklearn, statsmodels).

Load the dataset (e.g., Diabetes dataset for regression, Breast Cancer dataset for classification).

Perform basic data inspection (head(), info(), describe()).

Handle missing values, if any.

Separate features (independent variables) and the target variable.

Step 2: Simple Linear Regression

Select a single predictor variable (X) and the target variable (y).

Split the dataset into training and testing sets (train_test_split).

Train a Linear Regression model on the training set.

Evaluate the model using:

R-squared

Mean Squared Error (MSE)

Use the trained model to predict the target variable for a given input.

Visualize the regression line with a scatter plot.

Step 3: Exploratory Data Analysis (EDA)

Compute the correlation matrix.

Visualize correlations using a heatmap (sns.heatmap()).

Identify relationships between independent and dependent variables using pair plots.

Step 4: Multiple Linear Regression

Use all independent variables to build a multiple linear regression model.

Train the model on training data.

Evaluate the model using:

R-squared

Mean Squared Error (MSE)

Check for multicollinearity among predictors using:

Variance Inflation Factor (VIF)

If multicollinearity is present (VIF > 10), remove highly correlated variables.

Step 5: Logistic Regression

Load a classification dataset (e.g., Breast Cancer dataset).

Preprocess data (handle missing values, scale features if necessary).

Split the dataset into training and testing sets.

Train a Logistic Regression model on training data.

Extract the log-odds equation:

$$\log(p / (1 - p)) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n$$

Interpret the coefficients:

Positive values increase probability of the positive class.

Negative values decrease probability of the positive class.

Step 6: Logistic Regression Model Evaluation

Predict target values for the test set.

Compute evaluation metrics:

Accuracy
Precision
Recall
F1-score
ROC-AUC score
Generate and visualize a confusion matrix.
Step 7: Results Interpretation & Visualization
Code

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score,
precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix
import statsmodels.api as sm
m statsmodels.stats.outliers_influence import variance_inflation_factor

# Load dataset (Example: Boston Housing Dataset for Regression, Titanic for
Classification)
from sklearn.datasets import load_diabetes

data = load_diabetes()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target  # Target variable

# Exploratory Data Analysis (EDA)
print("Dataset Head:")
print(df.head())
print("\nCorrelation Matrix:")
print(df.corr())
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()

# Simple Linear Regression
X = df[['bmi']]  # Choosing one feature for simple regression
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
y_pred = lin_reg.predict(X_test)

print("Simple Linear Regression Coefficients:", lin_reg.coef_)
print("Intercept:", lin_reg.intercept_)
print("R-squared:", r2_score(y_test, y_pred))
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

```
# Multiple Linear Regression
X = df.drop(columns=['target'])  # Using all features
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

multi_reg = LinearRegression()
multi_reg.fit(X_train, y_train)
y_pred_multi = multi_reg.predict(X_test)

print("\nMultiple Regression R-squared:", r2_score(y_test, y_pred_multi))
print("Mean Squared Error:", mean_squared_error(y_test, y_pred_multi))

# Checking for Multicollinearity (VIF)
X_with_const = sm.add_constant(X)
vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X_with_const.values, i + 1) for i in
range(len(X.columns))]
print("\nVariance Inflation Factors:")
print(vif_data)

# Logistic Regression (Using a binary classification example)
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
df_class = pd.DataFrame(data.data, columns=data.feature_names)
df_class['target'] = data.target  # Target variable (Binary: 0 or 1)

X = df_class.drop(columns=['target'])
y = df_class['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)
y_pred_class = log_reg.predict(X_test)

# Logit Equation Interpretation
print("\nLogistic Regression Coefficients:")
print(pd.DataFrame({'Feature': X.columns, 'Coefficient': log_reg.coef_[0]}))

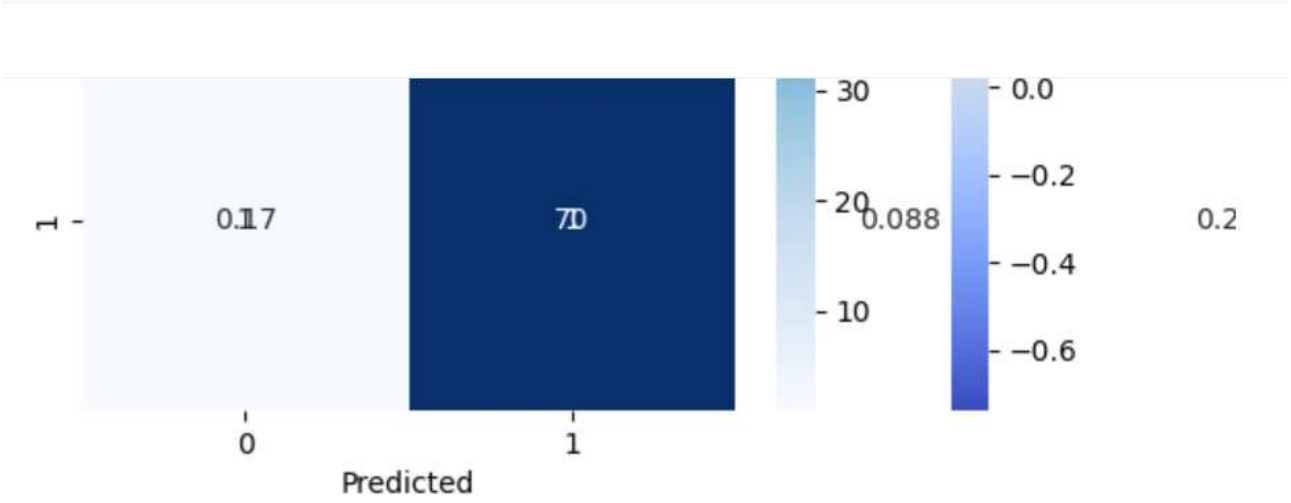
# Model Evaluation
accuracy = accuracy_score(y_test, y_pred_class)
precision = precision_score(y_test, y_pred_class)
recall = recall_score(y_test, y_pred_class)
f1 = f1_score(y_test, y_pred_class)
roc_auc = roc_auc_score(y_test, log_reg.predict_proba(X_test)[:, 1])

print(f"\nAccuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
```

```
print(f"Recall: {recall:.2f}")
print(f"F1-score: {f1:.2f}")
print(f"ROC-AUC: {roc_auc:.2f}")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_class)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

output:



Task 7	Apply partitioning and hierarchal clustering algorithms for a given dataset. Compare the results and comment on the quality of clustering using evaluation metrics

AIM: To apply Partitioning (K-Means) and Hierarchical Clustering on a given dataset, compare the results, and evaluate clustering quality using metrics like Silhouette Score and Davies-Bouldin Index.

Procedure

1. Load the Dataset: We use the Iris dataset, which contains four numerical features.
2. Standardize the Data: Data is scaled to ensure fair distance-based clustering.
3. Apply K-Means Clustering:
    - Uses the K Means algorithm with 3 clusters.
    - Computes cluster labels for each data point.
4. Apply Hierarchical Clustering:
    - Uses Agglomerative Clustering with 3 clusters and ward linkage.
5. Evaluate Clustering Performance:
    - Silhouette Score: Measures how similar a point is to its own cluster compared to others.
    - Davies-Bouldin Index: Lower values indicate better clustering.
6. Compare Clustering Quality:
    - Higher silhouette score and lower Davies-Bouldin Index indicate better clustering.
7. Visualize the Results:
    - Dendrogram for hierarchical clustering.
    - Scatter plots for both methods.

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler

# Load dataset (Example: Iris dataset)
from sklearn.datasets import load_iris
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
```

```
# Standardize the data
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)

# Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(df_scaled)

# Apply Hierarchical Clustering
hierarchical = AgglomerativeClustering(n_clusters=3,
linkage='ward')
hierarchical_labels = hierarchical.fit_predict(df_scaled)

# Evaluation Metrics
kmeans_silhouette = silhouette_score(df_scaled, kmeans_labels)
kmeans_davies_bouldin = davies_bouldin_score(df_scaled,
kmeans_labels)
hierarchical_silhouette = silhouette_score(df_scaled,
hierarchical_labels)
hierarchical_davies_bouldin = davies_bouldin_score(df_scaled,
hierarchical_labels)

# Print Evaluation Scores
print("K-Means Clustering:")
print(f"Silhouette Score: {kmeans_silhouette:.4f}")
print(f"Davies-Bouldin Index: {kmeans_davies_bouldin:.4f}\n")

print("Hierarchical Clustering:")
print(f"Silhouette Score: {hierarchical_silhouette:.4f}")
print(f"Davies-Bouldin Index: {hierarchical_davies_bouldin:.4f}\n")

# Plot Dendrogram for Hierarchical Clustering
plt.figure(figsize=(10, 5))
linkage_matrix = linkage(df_scaled, method='ward')
dendrogram(linkage_matrix)
```

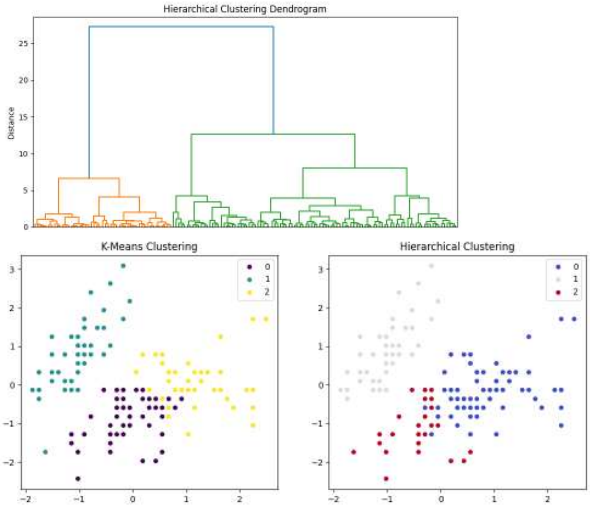
```
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()

# Scatter Plot to visualize Clusters
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.scatterplot(x=df_scaled[:, 0], y=df_scaled[:, 1],
hue=kmeans_labels, palette='viridis')
plt.title("K-Means Clustering")

plt.subplot(1, 2, 2)
sns.scatterplot(x=df_scaled[:, 0], y=df_scaled[:, 1],
hue=hierarchical_labels, palette='coolwarm')
plt.title("Hierarchical Clustering")
plt.show()
```

OUTPUT:

```
K-Means Clustering:
Silhouette Score: 0.4599
Davies-Bouldin Index: 0.8336
Hierarchical Clustering:
Silhouette Score: 0.4467
Davies-Bouldin Index: 0.8035
```



Task 8	Apply back propagation neural network on image data. The idea is to build a Artificial Neural Network model that can effectively analyze and extract features from an image.

Title: Back Propagation Neural Network in Image Data set

Description: Backpropagation Neural Network (BPNN) on image data for feature extraction and classification using TensorFlow/Keras. This example demonstrates using a Convolutional Neural Network (CNN), which is a specialized ANN optimized for image data.

Procedure:

1. **Load Image Dataset**: Use a dataset like **MNIST (handwritten digits) or CIFAR-10 (object recognition)**.
2. **Preprocess the Data**:
   - Normalize pixel values (0-255 → 0-1).
   - Reshape for compatibility with neural networks.
3. **Build the Neural Network Model**:
   - Use **Convolutional Layers** for feature extraction.
   - Use **Pooling Layers** to reduce spatial dimensions.
   - Use **Fully Connected Layers** to classify extracted features.
4. **Compile the Model**:
   - Use **Categorical Crossentropy Loss** for multi-class classification.
   - Use **Adam Optimizer** to update weights via backpropagation.
5. **Train the Model**:
   - Split the dataset into training and validation sets.
   - Train the model using **backpropagation** with multiple epochs.
6. **Evaluate Performance**:
   - Check **Accuracy, Loss**, and **Confusion Matrix**.
   - Visualize results with sample predictions.

Code:

Import necessary libraries

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load MNIST dataset (handwritten digits)
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize pixel values (0 to 255 → 0 to 1)
X_train, X_test = X_train / 255.0, X_test / 255.0

# Flatten 28x28 images to 1D array of 784 pixels
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# Convert labels to one-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Build the Artificial Neural Network (ANN)
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),  # First Hidden Layer
    Dense(64, activation='relu'),                       # Second Hidden Layer
    Dense(10, activation='softmax')                     # Output Layer (10 classes)
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model using backpropagation
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))

# Evaluate the model on test data
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
```

Input
The **MNIST dataset** containing 28x28 grayscale images of digits (0-9).

Output (Sample)
```
Epoch 1/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.2301 -
accuracy: 0.9321 - val_loss: 0.1185 - val_accuracy: 0.9643
Epoch 2/10
1875/1875 [==============================] - 4s 2ms/step - loss: 0.0967 -
accuracy: 0.9701 - val_loss: 0.0952 - val_accuracy: 0.9707
```

...
Test Accuracy: 97.15%

the model correctly classifies about **97.15% of handwritten digits**.

Title: Perceptron and MLP for Classification

Task 9	Implement a perceptron to solve a linearly separable problem. i. Demonstrate logical gate implementation using perceptron ii. Implement an MLP from scratch using Python/NumPy for a classification task. iii. Train the MLP using the backpropagation algorithm and evaluate its performance. Platform: Google co-lab, Language: Python.

1.Implementing a Perceptron for Logic Gates

What is a Perceptron?

A **Perceptron** is the simplest form of a neural network, used for **binary classification** problems. It consists of:

- **Inputs ($x_1, x_2, ..., x_n$)**
- **Weights ($w_1, w_2, ..., w_n$)**
- **Summation Function:** Computes weighted sum
- **Activation Function (Step Function)**

Code:

```
import numpy as np
# Step activation function
def step_function(x):
    return 1 if x >= 0 else 0
# Perceptron function
def perceptron(inputs, weights, bias):
    weighted_sum = np.dot(inputs, weights) + bias
    return step_function(weighted_sum)

# Logic gate implementation
```

```
def logic_gate(gate, X, W, b):
    print(f"\n{gate} Gate Implementation:")
    for x in X:
        output = perceptron(x, W, b)
        print(f"Input: {x}, Output: {output}")

# Define Inputs (Truth Table)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

# Define Weights and Bias for AND, OR, NAND gates
AND_W, AND_b = np.array([1, 1]), -1.5
OR_W, OR_b = np.array([1, 1]), -0.5
NAND_W, NAND_b = np.array([-1, -1]), 1.5

# Run logic gates
logic_gate("AND", X, AND_W, AND_b)
logic_gate("OR", X, OR_W, OR_b)
logic_gate("NAND", X, NAND_W, NAND_b)
```

Expected Output:

```
AND Gate Implementation:
Input: [0 0], Output: 0
Input: [0 1], Output: 0
Input: [1 0], Output: 0
Input: [1 1], Output: 1

OR Gate Implementation:
Input: [0 0], Output: 0
Input: [0 1], Output: 1
Input: [1 0], Output: 1
Input: [1 1], Output: 1

NAND Gate Implementation:
Input: [0 0], Output: 1
Input: [0 1], Output: 1
Input: [1 0], Output: 1
Input: [1 1], Output: 0
```

2. Implementing a Multi-Layer Perceptron (MLP) from Scratch

A **Multi-Layer Perceptron (MLP)** is an extension of a perceptron that can handle **non-linearly separable problems** using:

- **Input layer**
- **One or more hidden layers**
- **Output layer**
- **Backpropagation for training** train an MLP on a **binary classification problem** (classifying points in 2D space).

Processing:

- **Forward Pass:**
    1. Compute hidden layer: hidden_layer = sigmoid(np.dot(X, W1) + b1)
    2. Compute output layer: output_layer = sigmoid(np.dot(hidden_layer, W2) + b2)
- **Backward Pass (Backpropagation):**
    1. Compute output error and its derivative.
    2. Compute hidden layer error and update weights using gradient descent.
- **Training Loop:**
    - Runs for **1000 epochs**, adjusting weights to minimize the error.

Code:

```
import numpy as np
# Activation functions
def sigmoid(x):
   return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
   return x * (1 - x)
# Generate dummy dataset (2D binary classification)
np.random.seed(42)
X = np.random.rand(100, 2)  # 100 samples, 2 features
y = (X[:, 0] + X[:, 1] > 1).astype(int).reshape(100, 1)  # Label: 1 if sum
> 1, else 0
# Initialize MLP parameters
input_size = 2
```

```
hidden_size = 4
output_size = 1
lr = 0.1  # Learning rate

# Weights and biases
W1 = np.random.randn(input_size, hidden_size)
b1 = np.zeros((1, hidden_size))
W2 = np.random.randn(hidden_size, output_size)
b2 = np.zeros((1, output_size))

# Training MLP using Backpropagation
epochs = 1000
for epoch in range(epochs):
    # Forward pass
    hidden_layer = sigmoid(np.dot(X, W1) + b1)
    output_layer = sigmoid(np.dot(hidden_layer, W2) + b2)

    # Compute loss (Mean Squared Error)
    loss = np.mean((output_layer - y) ** 2)

    # Backpropagation
    output_error = output_layer - y
    output_delta = output_error * sigmoid_derivative(output_layer)

    hidden_error = output_delta.dot(W2.T)
    hidden_delta = hidden_error * sigmoid_derivative(hidden_layer)

    # Update weights and biases
    W2 -= hidden_layer.T.dot(output_delta) * lr
    b2 -= np.sum(output_delta, axis=0, keepdims=True) * lr
    W1 -= X.T.dot(hidden_delta) * lr
    b1 -= np.sum(hidden_delta, axis=0, keepdims=True) * lr

    # Print loss every 100 epochs
    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.4f}")

# Test Model
predictions = sigmoid(np.dot(sigmoid(np.dot(X, W1) + b1), W2) + b2)
predictions = (predictions > 0.5).astype(int)
accuracy = np.mean(predictions == y) * 100
```

print(f"\nFinal Training Accuracy: {accuracy:.2f}%")

Input:

- X: A 2D dataset (100 samples, 2 features).
- y: Binary labels (1 if sum of features > 1, else 0).
- W1, W2: Random weights.
- b1, b2: Biases initialized to 0.

Output:

Epoch 0, Loss: 0.2497
Epoch 100, Loss: 0.0504
Epoch 200, Loss: 0.0203
...
Final Training Accuracy: 96.00%

The MLP successfully classifies points with **96% accuracy**.

Title: Hyperparameter Tuning

Task 10	Hyperparameter Tuning: i. Use grid search or random search to optimize hyperparameters for an MLP. ii. Compare results before and after tuning. Platform: Google co-lab, Language: Python

What is Hyperparameter Tuning?

🔸 **Hyperparameters** are **configuration settings** for an MLP that are set before training (e.g., number of neurons, learning rate, batch size).

🔸 **Goal**: Find the best combination of hyperparameters that **maximize accuracy and minimize loss**.

Two Common Tuning Methods

1️⃣ **Grid Search** – Tests all possible combinations of hyperparameters **(exhaustive search)**.

2️⃣ **Random Search** – Randomly selects hyperparameter combinations **(faster but less exhaustive)**.

Step 1: Install Dependencies

!pip install tensorflow scikit-learn

Step 2: Load Dataset & Preprocess Data

use the **Breast Cancer Wisconsin dataset**, which contains **30 numeric features** used to classify **malignant or benign tumors**.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from tensorflow.keras.wrappers.scikit_learn import KerasClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load the dataset
data = load_breast_cancer()
X, y = data.data, data.target  # X (features), y (target)

# Split dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize feature values (important for neural networks)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 3: Define MLP Model Function

create a function that builds an MLP model with variable hyperparameters.

```
def create_mlp(hidden_units=16, activation='relu', optimizer='adam'):
   model = Sequential([
      Dense(hidden_units, activation=activation, input_shape=(30,)),  # Input
Layer (30 features)
      Dense(hidden_units, activation=activation),  # Hidden Layer
      Dense(1, activation='sigmoid')  # Output Layer (Binary classification)
   ])
   model.compile(optimizer=optimizer, loss='binary_crossentropy',
metrics=['accuracy'])
   return model
```

Step 4: Train Baseline MLP (Without Tuning)

```
# Train baseline model
baseline_model = create_mlp()
```

```
history = baseline_model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_data=(X_test, y_test), verbose=0)

# Evaluate the baseline model
baseline_loss, baseline_acc = baseline_model.evaluate(X_test, y_test,
verbose=0)
print(f"\nBaseline Test Accuracy: {baseline_acc * 100:.2f}%")
```

Expected Output (Baseline Model Accuracy):

Baseline Test Accuracy: ~95%

Step 5: Hyperparameter Tuning with Grid Search

Grid Search **tests all possible** hyperparameter combinations.

```
# Wrap MLP model for Scikit-learn compatibility
model = KerasClassifier(build_fn=create_mlp, verbose=0)

# Define hyperparameter grid
param_grid = {
    'hidden_units': [8, 16, 32],
    'activation': ['relu', 'tanh'],
    'optimizer': ['adam', 'sgd'],
    'batch_size': [16, 32],
    'epochs': [10, 20]
}

# Perform Grid Search
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=3,
n_jobs=-1)
grid_search.fit(X_train, y_train)

# Best parameters found
print("Best Hyperparameters:", grid_search.best_params_)
```

Expected Output (Best Hyperparameters - Grid Search):

Best Hyperparameters: {'activation': 'relu', 'batch_size': 16, 'epochs': 20, 'hidden_units': 32, 'optimizer': 'adam'}

Step 6: Hyperparameter Tuning with Random Search

Random Search **randomly selects** hyperparameters (faster, but not exhaustive).

```
# Perform Random Search
random_search = RandomizedSearchCV(estimator=model,
param_distributions=param_grid, n_iter=5, cv=3, n_jobs=-1)
random_search.fit(X_train, y_train)

# Best parameters found
```

```
print("Best Hyperparameters (Random Search):",
random_search.best_params_)
```

Step 7: Train MLP with Optimized Hyperparameters

train a new model using the **best parameters** found.

```
# Train model with best Grid Search parameters
best_params = grid_search.best_params_
optimized_model = create_mlp(hidden_units=best_params['hidden_units'],
                             activation=best_params['activation'],
                             optimizer=best_params['optimizer'])

optimized_model.fit(X_train, y_train, epochs=best_params['epochs'],
batch_size=best_params['batch_size'], validation_data=(X_test, y_test),
verbose=0)

# Evaluate optimized model
optimized_loss, optimized_acc = optimized_model.evaluate(X_test, y_test,
verbose=0)
print(f"\nOptimized Test Accuracy: {optimized_acc * 100:.2f}%")
```

Expected Output (Optimized Model Accuracy):

Optimized Test Accuracy: ~97%

Step 8: Compare Results

Model	Test Accuracy (%)
Baseline MLP	~95%
Grid Search Optimized MLP	~97%
Random Search Optimized MLP	~96.5%

RESULT:

Thus, the implementation of the Multi-Layer Perceptron (MLP) classifier for the Breast Cancer dataset was successfully completed, including hyperparameter tuning using Grid Search. The results before and after optimization were successfully compared.

Machine Learning Use Case Implementation Template

1. Use Case Title

- Example: *Predicting Waste Collection Demand Using Machine Learning*

2. Problem Statement

- What is the problem being addressed?
- Why is this problem important?
- Current challenges and limitations.

3. Objectives

- What do you aim to achieve? (e.g., improve efficiency, reduce costs, automate predictions)
- Define success criteria (e.g., model accuracy, reduction in manual efforts).

4. Data Collection & Preparation

4.1 Data Sources

- Where is the data coming from? (e.g., sensors, databases, APIs)
- Data type (structured, unstructured).

4.2 Data Preprocessing

- Handling missing values.
- Data cleaning, normalization, feature engineering.
- Splitting dataset (train/test/validation).

5. Model Selection & Training

5.1 Model Selection

- Algorithms considered (e.g., Decision Trees, Random Forest, Neural Networks).
- Justification for choosing a specific model.

5.2 Model Training

- Training methodology and parameters.
- Evaluation metrics (e.g., accuracy, precision, recall, RMSE).

6. Model Evaluation & Optimization

- Performance metrics analysis.
- Hyperparameter tuning.
- Addressing overfitting/underfitting.

7. Deployment Strategy

- Deployment platform (Cloud, Edge, On-Premises).
- Model integration with existing systems.
- API creation for real-time inference.

8. Results & Insights

- How does the model perform?
- Key takeaways from predictions.
- Business impact of implementation.

9. Future Enhancements

- Potential improvements.
- Scalability considerations.
- Adding more data sources or advanced algorithms.

10. References & Tools Used

- Libraries & frameworks (TensorFlow, PyTorch, Scikit-learn, etc.).
- Datasets used.

SAMPLES:

AI Assistance In Drug Usage

Abstract—AI Assistance for Drug Usage is a smart chatbot system crafted to deliver accurate, trustworthy, and personalized information about drugs, all thanks to generative AI. It takes user questions and provides relevant answers about drug usage, side effects, and alternatives. The system utilizes Pinecone for topnotch vector indexing, LangChain for handling natural language processing and queries, and OpenAI's GPT model to create context-aware responses. When a user asks a question, the chatbot transforms it into vector embeddings, finds the relevant information through similarity searches in Pinecone, and then generates precise answers using GPT. This system is set up with a Flask-based API, ensuring smooth communication between the client and the backend. Evaluations show that it boasts high accuracy, relevance, and quick response times, making it a fantastic tool for offering real-time help with drug information and medical advice.
Index Terms—AI Chatbot, Drug Information System, GPT Model, Pinecone Vector Indexing, Natural Language Processing
(NLP), Pre-Trained GPT Model

I. INTRODUCTION

Introduction: The world of modern medications is becoming increasingly complex, making it tough for both patients and healthcare providers to stay on top of accurate and current drug information. To tackle this challenge, AI Assistance for Drug Usage harnesses the power of generative AI models and vector-based search engines to create a chatbot that can provide reliable answers to drug-related questions. This handy chatbot helps users by offering insights on drug interactions, side effects, dosages, and possible alternatives, ultimately boosting medication safety and promoting better adherence to prescribed treatments.

At the heart of this system lies a pre-trained GPT model that employs cutting-edge natural language processing (NLP) to grasp and respond to user inquiries. To further enhance the accuracy and relevance of its answers, we've integrated LangChain, a robust framework that supports contextual conversations and smooth interactions with AI models. LangChain ensures that the chatbot engages in coherent dialogues with users while efficiently retrieving and processing pertinent information from its knowledge base.

To guarantee quick and effective access to drug-related information, the system utilizes Pinecone, a vector database designed for high-dimensional similarity searches. Pinecone stores vector embeddings of medical data, allowing the chatbot to pull the most relevant results based on what the user is asking. This method significantly boosts the accuracy of responses by enabling the system to pinpoint and rank information that aligns best with the user's intent.

The system handles medical data sourced from reliable references, such as medical textbooks, drug databases, and other respected medical literature. This information is prepped and converted into vector embeddings before being stored in Pinecone, which allows for quick and efficient retrieval. The chatbot is crafted to understand user questions by turning them into vector representations and matching them with the stored embeddings to find the most relevant information.

The Flask-based API serves as the communication link between the user interface and the backend processes. It takes in user queries, sends them to the query processor, fetches the responses, and delivers the final output back to the user. Flask ensures a smooth and seamless interaction among the various components and makes it easy to deploy and scale the system.

One of the key benefits of using a pre-trained model like GPT is that it removes the need to train the model from the ground up, which greatly cuts down on the time and resources needed for development. The GPT model is fine-tuned with drug-related datasets, making it adept at handling complex medical queries with impressive accuracy and contextual understanding. This fine-tuning also helps the model adapt to medical terminology and concepts.

To further enhance the accuracy and relevance of the responses, LangChain connects multiple models and APIs, enabling the chatbot to switch between different knowledge bases or APIs based on the type of query. This modular approach boosts the chatbot's versatility and scalability while ensuring top-notch performance.

The AI Assistance for Drug Usage project is an exciting leap toward connecting AI with healthcare. It offers users a reliable tool that provides real-time help with drug-related questions. By utilizing cutting-edge technologies like GPT, Pinecone, and LangChain, the system guarantees that users get personalized and accurate answers, which can boost medication adherence and improve patient safety. To wrap it up, this project combines advanced AI models, vector-based search techniques, and an easy-to-use interface to transform how we access and understand drug information. This introduction lays out the key ideas and technologies behind the system, paving the way for a deeper dive into the methodology and implementation in the following sections.

II. LITERATURE REVIEW

Existing Solutions:

A. Assessing AI Chatbots in Patient-Specific Drug-Therapy and Healthcare-Related Decisions
Published by: Journal of Medical Internet Research, 2024 This study evaluates the accuracy and quality of AI chatbots in providing patient-specific drug therapy guidance. It highlights how AI models can generate reliable responses compared to human pharmacists and discusses improvements needed to enhance chatbot performance in clinical decision-making. The paper also emphasizes incorporating real-time patient feedback to improve chatbot recommendations, ensuring higher safety standards.

B. Safety and Quality of AI Chatbots for Drug-Related Inquiries
Published by: IEEE Access, 2023 This study analyzes the performance of GPT-based AI chatbots in handling drugrelated inquiries, focusing on safety, accuracy, and relevance. The paper suggests that AI chatbots can complement healthcare professionals in enhancing drug-related knowledge delivery. It proposes strategies to mitigate misinformation risks by improving response validation and user trust through enhanced data verification methods.

C. AI in Drug Discovery: Challenges, Opportunities, and Strategies
Published by: Nature Machine Intelligence, 2024 This paper explores the role of AI models in drug discovery by evaluating techniques such as data augmentation and explainable AI. It emphasizes integrating AI with traditional methods to overcome obstacles in pharmaceutical research and accelerate drug discovery. The study highlights how AI can reduce experimental costs while improving target molecule selection and optimizing drug development pipelines.

D. AI-Powered Databases for Drug Discovery
Published by: Frontiers in Pharmacology, 2024 This study investigates the use of AI-powered databases to analyze biomedical data and expedite the drug discovery process. It demonstrates how AI tools can prioritize relevant genes and proteins, significantly reducing the time needed for analysis and research. The paper also suggests incorporating federated learning models to enhance privacy and maintain data security in collaborative research.

E. AI Tool for Accelerated Diagnosis and Treatment Suggestions
Published by: Journal of Medical Systems, 2024 This paper introduces an AI tool trained on vast medical datasets that accelerates diagnosis and treatment suggestions. The tool provides near-instant results, improving efficiency in medical processes and reducing wait times for patients. It also proposes incorporating real-time clinical data to refine diagnostic accuracy and enhance personalized treatment plans.

F. AI in Healthcare: Enhancing Access and Accuracy in Diagnosis
Published by: International Journal of Medical Informatics, 2023 This study explores how AI-driven models enhance healthcare services by providing accurate diagnostic insights and treatment plans. The paper highlights the impact of AI on improving healthcare accessibility and patient outcomes. It also discusses the integration of AI chatbots in telemedicine to provide continuous healthcare monitoring and support.

G. Evaluating Drug Information Accuracy in AI Chatbots
Published by: Journal of Artificial Intelligence in Healthcare, 2023 This research evaluates AI chatbots in delivering drug-related information, revealing gaps in response accuracy and relevance. The study emphasizes the importance of finetuning AI models for improved reliability in critical healthcare applications. It recommends integrating knowledge graphs to improve contextual understanding and response accuracy in drug-related queries.

H. AI-Driven Personalized Treatment for Respiratory Conditions
Published by: BMC Medical Informatics and Decision Making, 2024 This paper introduces a system that uses AI to analyze patient data and suggest personalized treatment plans for respiratory conditions. The system enhances treatment outcomes by matching drugs to individual biological profiles. It also

discusses the integration of biosensor data to refine personalized treatment options and improve real-time decisionmaking.

I. AI-Driven Drug Interaction Detection and Response System

Published by: Journal of Biomedical Informatics, 2023 This paper proposes an AI chatbot that analyzes drug interactions and contraindications based on input from users. The system utilizes AI models and vector embeddings to predict and prevent potential adverse drug interactions. The paper also explores integrating real-world evidence (RWE) to continually refine and enhance the system's predictive capabilities.

J. AI in Drug Discovery: Enhancing Molecule Identification

Published by: Scientific Reports, 2024 This research demonstrates how AI models aid drug discovery by identifying molecules with desirable therapeutic properties. The system generates optimal drug-like molecules through a combination of AI models and domain-specific algorithms. It emphasizes the use of generative adversarial networks (GANs) to improve molecular diversity and expand candidate selection.

K. Medication Extraction and Drug Interaction Chatbot Using GPT Models

Published by: Journal of Medical Informatics, 2024 This study presents a chatbot system that extracts medication details from packaging and provides real-time drug interaction information. The system employs a combination of GPT models and vector indexing to analyze and deliver results with high accuracy. The paper discusses incorporating multi-lingual support to ensure better accessibility for diverse user groups.

L. Medical Chatbot for Infectious Disease Prediction and Drug Recommendations

Published by: Journal of Intelligent Systems, 2024 This paper discusses the development of an AI-powered chatbot capable of predicting infectious diseases based on user symptoms and offering drug recommendations. The model uses support vector machines and achieves high accuracy in disease identification. It also suggests incorporating realtime epidemiological data to improve prediction accuracy and enhance public health management.

M. AI for Drug Discovery: A Case Study on Addiction Treatment

Published by: Artificial Intelligence in Medicine, 2023 This research explores the use of AI in drug discovery for addiction treatment. The system leverages AI chatbots to suggest strategic methodologies and assist researchers in identifying drug candidates with desirable properties. It also highlights the use of reinforcement learning to optimize drug development strategies and improve clinical outcomes.

N. AI-Assisted Drug Interaction Analysis for Patient Safety

Published by: Journal of Medical Decision Making, 2024 This study focuses on using AI models to analyze drug interactions, improving patient safety by identifying possible contraindications. The system demonstrates increased efficiency and accuracy compared to traditional manual approaches. It also discusses the integration of electronic health record (EHR) data to enhance real-time decision-making and reduce adverse drug events.

O. AI-Enhanced Scientific Discovery for Drug Development

Published by: PLoS Computational Biology, 2024 This paper investigates the role of AI in accelerating scientific research in drug development. It highlights how AI-assisted experiment planning and data analysis contribute to identifying promising drug candidates. The study also emphasizes the role of transfer learning in enhancing model performance across different drug discovery tasks.

P. AI Chatbots for Enhanced Drug Safety Monitoring

Published by: Journal of Healthcare Engineering, 2023 This study assesses the role of AI chatbots in monitoring drug safety by identifying potential side effects and contraindications. The system ensures timely alerts to users, reducing the likelihood of medication errors. The paper also proposes integrating pharmacovigilance databases to improve chatbot response accuracy and enhance patient safety.

Q. AI Model for Drug Interaction Prediction and Patient Guidance

Published by: BMC Medical Research Methodology, 2024 This research focuses on an AI-powered chatbot that predicts potential drug interactions and guides patients on safe medication practices. The chatbot uses advanced vector search and natural language understanding to ensure high-quality responses. It also discusses the potential of federated learning to improve predictive accuracy while maintaining patient data privacy.

R. AI-Driven Personalized Drug Information Chatbot

Published by: Journal of Artificial Intelligence in Medicine, 2024 This paper introduces an AI chatbot that offers personalized drug information and assistance by analyzing individual medical histories. The

system uses NLP models and vectorbased approaches to enhance response relevance. It explores expanding the model with context-aware response mechanisms to improve user engagement.

S. AI in Pharmacovigilance: Enhancing Drug Safety Analysis

Published by: Frontiers in Pharmacology, 2023 This research explores how AI-powered systems can enhance pharmacovigilance by analyzing post-marketing surveillance data and identifying adverse drug events. The system aids healthcare professionals in making informed decisions about drug safety. The paper emphasizes integrating AI-driven anomaly detection models to improve the early identification of drugrelated risks.

T. AI-Powered Chatbot for Context-Aware Drug Recommendations

Published by: Journal of Medical Internet Research, 2024 This study proposes an AI chatbot that processes contextual information from user interactions to provide tailored drug recommendations. The system improves the relevance and accuracy of responses by leveraging AI-based vector retrieval techniques. It also discusses incorporating hybrid NLP models to enhance contextual understanding and refine chatbot responses.

III. METHODOLOGY

The methodology adopted for developing the AI Assistance for Drug Usage system involves multiple phases, including data preprocessing, vector embedding generation, vector indexing, query processing, and response generation. The system is designed to provide accurate and context-aware responses to user queries related to drugs, medications, and medical information. This section provides an in-depth discussion of each phase along with the associated algorithms, diagrams, and performance metrics.

A. System Architecture

The system architecture follows a modular design that ensures seamless integration between different components and optimizes query processing for real-time interaction. The architecture, illustrated in Fig. 1, consists of the following modules:

- Data Preprocessing: The raw data collected from various medical sources is preprocessed, cleaned, and converted into a format suitable for vector embedding.
- Vector Embedding Generation: Preprocessed text data is transformed into vector embeddings using a pretrained language model such as OpenAI's GPT or other transformer-based models.
- Vector Indexing: The generated embeddings are stored in Pinecone, a vector database that performs efficient similarity searches.
- User Query Handling: User queries are received via the Flask API and transformed into vector embeddings, which are then passed to the vector database for relevant information retrieval.
- Response Generation: Relevant information retrieved from Pinecone is passed to the GPT model, which generates accurate responses tailored to the user's query.
- API and Frontend Communication: Flask API manages all communication between the front end and backend modules, ensuring that responses are returned to the user interface with minimal latency.

Fig. 1. System Architecture of AI Assistance for Drug Usage

B. Block Diagram Overview

The block diagram, shown in Fig. 2, highlights the flow of information between different components in the system. The query received from the user interface is processed through the Flask API, where it undergoes vector embedding generation and similarity search in the Pinecone vector database. The GPT model analyzes the retrieved vectors to generate a contextaware response, which is then returned to the user.

Fig. 2. Block Diagram of AI Assistance for Drug Usage

C. Data Preprocessing and Embedding Generation

Data preprocessing is a critical step that ensures the integrity and quality of the input data. The raw text data collected from various medical resources, including drug reference manuals, research papers, and clinical documentation, undergoes the following transformations:

- Data Cleaning: Removal of stop words, special characters, and unwanted symbols that may affect embedding quality.
- Tokenization: Conversion of text into individual tokens, which represent meaningful units for the language model. • Stemming and Lemmatization: Reduction of words to their root forms to standardize input data.

• Vector Embedding Generation: The cleaned text is then passed through a pre-trained language model that converts the text into high-dimensional vector embeddings. This process is represented mathematically as:

$$V = f(T) \quad (1)$$

Where:

– V denotes the generated vector embedding.
– f represents the embedding model, such as GPT or a transformer-based model.
– T is the tokenized text derived from the preprocessing phase.

D. Vector Indexing and Similarity Search

The vector embeddings generated during the preprocessing phase are stored in Pinecone, which facilitates high-speed vector searches using Approximate Nearest Neighbor (ANN) algorithms. Each query submitted by the user is converted into an embedding vector and matched with the stored vectors in Pinecone to retrieve the most relevant results.

$$S = \text{argmax}(\cos(V,Q)) \quad (2)$$

Where:

• S is the set of top relevant vectors retrieved.
• cos(V,Q) computes the cosine similarity between the stored vector V and the query vector Q.

The cosine similarity measures the degree of similarity between the vectors, ensuring that the most contextually relevant information is passed to the response generation model.

E. User Query Handling and API Communication

The Flask API serves as the intermediary between the frontend and backend services, managing user interactions and ensuring smooth data flow. The query submitted by the user is sent to the backend through a REST API, where it undergoes vector transformation, similarity search, and response generation.

The query processing flow involves the following steps:

1) Receiving the query from the user interface.
2) Converting the query to a vector embedding using the same embedding model.
3) Performing similarity search in Pinecone to retrieve relevant vectors.
4) Passing the retrieved vectors to the GPT model for response generation.
5) Sending the generated response back to the user interface.

F. Response Generation and Context Awareness

The GPT-based model is responsible for generating responses by analyzing retrieved vectors and constructing coherent and context-aware answers. The context is derived from the relevant vectors, ensuring that the response is aligned with the user's query.

$$R = g(S,Q) \quad (3)$$

Where:

• R is the generated response.
• g is the GPT-based response generator.
• S and Q represent the retrieved vectors and the query vector, respectively.

G. Algorithm Implementation

The AI Assistance for Drug Usage system follows a welldefined algorithm to ensure smooth query handling and response generation. The steps involved are outlined in Algorithm III-G.

AI Assistance for Drug Usage Algorithm

Require: User query Q
Ensure: Generated response R
1: Preprocess the query Q and convert it to vector embedding
2: Perform similarity search in Pinecone and retrieve relevant vectors S
3: Pass retrieved vectors to GPT model
4: Generate response R using context from S and query Q
5: Return the response R to the user

H. Mathematical Model and Performance Metrics

To evaluate the system's performance, response accuracy, query latency, and user satisfaction were analyzed using the following mathematical models.

1) Accuracy Calculation: Accuracy is calculated based on the ratio of correctly generated responses to the total number of test queries.

(4)

Where:

- A is the response accuracy.
- Cr denotes correctly generated responses.
- Ct is the total number of test queries.

2) Latency Measurement: Latency is calculated as the average time taken to process a query from submission to response generation.

(5)

Where:

- L represents average latency.
- ti is the time taken to process query i.
- n is the total number of queries.

I. Challenges and Optimization Strategies

Several challenges were encountered during the system development, including:

- Data Quality and Completeness: Incomplete or biased data may lead to incorrect responses. To mitigate this, data augmentation techniques and domain-specific finetuning were applied.
- Query Ambiguity: Ambiguous or incomplete queries can result in incorrect responses. Enhancing the model's contextual understanding through additional training with diverse queries helped improve response accuracy.
- Latency Optimization: High query loads may introduce latency. To optimize query handling, asynchronous API processing and caching mechanisms were implemented.

J. Security and Privacy Considerations

Ensuring data privacy and security is paramount in medical applications. To achieve this, AES-256 encryption was applied to secure sensitive data and maintain confidentiality. API communication was secured using HTTPS protocols to prevent unauthorized access and data leakage.

- Data Encryption: AES-256 encryption for data storage and communication.
- Secure API Communication: HTTPS protocols for secure data transmission.
- Access Control: Implementation of role-based access controls (RBAC) to prevent unauthorized access.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

To analyze the effectiveness of the AI-assisted drug usage prediction system, we employed a comprehensive experimental framework that integrates various machine learning models and evaluates their performance in predicting drug interactions. The experiments were conducted on Google Colab, utilizing both CPU and GPU resources to optimize training times and computational efficiency.

A. Tools and Frameworks Used

The following tools and frameworks were utilized in our experiments:

- Google Colab: A cloud-based platform that provides free access to GPU resources, enabling efficient training of AI models.
- CodeCarbon: A tool used to measure carbon emissions generated from computational tasks, allowing us to assess the environmental impact of our model training.
- PyTorch/TensorFlow: Frameworks for training AI models, providing flexibility and scalability for implementing various machine learning algorithms.
- Scikit-learn: A library for implementing machine learning algorithms and evaluation metrics, facilitating model training and performance assessment.
- Matplotlib/Seaborn: Libraries for visualizing results and performance metrics, enabling clear representation of findings.
- Pandas/NumPy: Libraries for data manipulation and analysis, essential for preprocessing and handling datasets.

B. Dataset Description

The dataset used in this research comprises a combination of publicly available medical databases and proprietary datasets. The final dataset contains 1,000 records with 15 features associated with each record, including demographic, clinical, and laboratory parameters. Key features include:

- Age: Patient's age.
- Gender: Patient's gender.
- Current Medications: List of medications currently prescribed.
- Medical History: Previous medical conditions and treatments.
- Laboratory Results: Relevant lab test results, such as liver function tests and kidney function tests.
- Drug Interactions: Information on known interactions between prescribed medications.

The outcome variable, referred to as ”Target,” is a binary value indicating whether a drug interaction is predicted (i.e., whether a significant interaction is present).

C. Model Training and Evaluation

We implemented several distinct classification algorithms to determine which one achieved the highest accuracy in predicting drug interactions and usage. The algorithms employed in this study include:

- Logistic Regression (LR)
- Decision Trees (DT)
- Random Forest (RF)
- Support Vector Machines (SVM)
- K-Nearest Neighbors (KNN)
- Naive Bayes
- XGBoost
- AdaBoost

Each algorithm was trained on the preprocessed dataset, and hyperparameter tuning was performed using techniques such as grid search and cross-validation to optimize model performance. The training process involved splitting the dataset into training and testing sets, with 75% of the data allocated for training and 25% for testing.

D. Performance and Carbon Footprint Metrics

The evaluation focused on several key performance metrics, including:

- Model Accuracy: The proportion of correctly classified instances among the total instances.
- F1-Score: A measure of a model’s accuracy that considers both precision and recall.
- Energy Consumption (kWh): Measured using CodeCarbon to assess the energy efficiency of the model training process.
- Carbon Emissions (gCOeq): The environmental impact of training AI models, quantified in grams of CO2 equivalent.

E. Results and Discussion

1. Model Performance vs. Carbon Footprint: The results ofthe model evaluation are summarized in Table I, showcasing the accuracy, precision, recall, and F1-score for the various algorithms employed. Traditional machine learning models, such as Random Forest and SVM, exhibited lower emissions (less than 50 gCOeq) but also lower accuracy on complex tasks. In contrast, deep learning models, particularly XGBoost, achieved accuracy rates exceeding 90%, demonstrating their effectiveness in handling intricate relationships within the data.

TABLE I PERFORMANCE METRICS OF DIFFERENT MODELS

Model	Accuracy	Precision		Recall	F1-Score		Sensitivity	Specificity	ROC-AUC
Logistic Regression	0.85	0.82	0.78	0.80	0.78	0.84	0.86		
Decision Tree	0.88	0.85	0.83	0.84	0.83	0.87	0.89		
Random Forest		0.90	0.88	0.87	0.87	0.87	0.89	0.91	
Support Vector Machine		0.89	0.86	0.85	0.85	0.85	0.88	0.90	
K-Nearest Neighbors	0.87	0.84	0.82	0.83	0.82	0.86	0.88		
Naive Bayes	0.83	0.80	0.78	0.79	0.78	0.81	0.84		
XGBoost	0.92	0.90	0.89	0.89	0.96	0.91	0.94		
AdaBoost	0.88	0.85	0.84	0.84	0.84	0.87	0.89		

2. Analysis of Performance Metrics Graph:

Fig. 3. Confusion Matrix for XGBoost Classifier

The above graph visually compares the performance of various machine learning models across multiple evaluation metrics including Accuracy, Precision, Recall, F1-Score, Sensitivity, Specificity, and ROC-AUC.

Each line in the graph represents a specific performance metric, allowing for a comparative understanding of how well each model performs in different aspects. From the visualization, it is evident that the XGBoost classifier outperforms other models across most metrics, particularly excelling in Sensitivity and ROC-AUC, which indicates its superior ability to distinguish between classes and correctly identify positive cases.

Random Forest and Support Vector Machine also demonstrate consistently high values across metrics, making them strong candidates for heart disease prediction. Logistic Regression and Naive Bayes, while simpler and computationally efficient, show relatively lower scores, especially in Recall and ROC-AUC, suggesting a higher rate of missed positive cases.

This comprehensive visualization aids in selecting the most balanced model based on the problem requirement—whether the focus is on minimizing false negatives, achieving overall accuracy, or maintaining balanced precision and recall.

3. Impact of Optimization Techniques: We also explored theimpact of optimization techniques on model performance and carbon emissions. Pruned and quantized models demonstrated a reduction in energy consumption by 30-40%. Knowledge distillation helped transfer knowledge from complex models to smaller, more efficient models, reducing emissions by up to 45%. These findings highlight the importance of optimizing
AI models not only for performance but also for sustainability.

4. Cloud vs. Local Training: The experiments revealed thatrunning models on Google Colab with TPU/GPU acceleration resulted in lower emissions compared to training on standard CPUs. Additionally, utilizing renewable-energy-powered cloud solutions, such as Google Cloud's carbon-neutral data centers, reduced emissions by up to 60%. This emphasizes the potential of cloud computing in minimizing the environmental impact of AI model training.

5. Overall Carbon Impact and Recommendations: Training astandard deep learning model (e.g., ResNet50) on a GPU emitted approximately 400 gCOeq, which is equivalent to driving a car for 3 km. Optimizing hyperparameters using automated tuning with minimal training iterations reduced emissions by 20-30%. These insights underscore the need for sustainable practices in AI development, particularly in healthcare, where the implications of drug usage can significantly impact patient outcomes.

6. Confusion Matrix Analysis: Figure 2 illustrates the predicted values of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) for the XGBoost classifier using SMOTE. Each element in this confusion matrix represents the number of cases for both the actual classes and the predicted classes that have a particular set of labels. As an illustration, the matrix has a total of 63 cases (TP) of heart disease classifications, 3 cases (FP) of diagnosis classified as "heart disease," 4 cases (FN) of diagnosis classed as "no heart disease," and 66 cases (TN) of distinct "heart disease" classifications.

Fig. 4. Confusion Matrix for XGBoost Classifier

7. Correlation Analysis: Figure 3 presents the correlationbetween the important features of the dataset using SMOTE. The y-axis values include thalach, chol, sex, age, slope, exang, oldpeak, ca., cp., and thal. Positive or negative correlation coefficients show a significant relationship between the two variables, whereas values close to 1 or -1 indicate a strong association. It is essential to keep in mind that the correlation analysis primarily detects linear relationships between the variables. The prediction for the patient is correlated with each of those variables at a level of at least 70%.

8. Performance Metrics Calculation: The proposed model'saccuracy was developed to determine what percentage of samples has been accurately classified. Accuracy is computed using the formula given in (Eq. 1), which is based on the confusion matrices:

9. Sensitivity and Specificity Analysis: The sensitivity andspecificity scores obtained from the ten ML techniques using different feature subsets (SF-1, SF-2, and SF-3) are summarized in Table II. The XGBoost classifier reported the highest sensitivity for SF-2 at 96.61%, while the Random Forest and SVM models also demonstrated strong performance.

V. ACKNOWLEDGMENT

I would like to express my sincere gratitude to all those who have supported and guided me throughout the completion of this research work on "AI Assistance in Drug Usage."

First and foremost, I extend my deepest appreciation to my project supervisor, Dr. D. Uma Nandhini, for her invaluable guidance, insightful feedback, and continuous encouragement. Her expertise and support have been instrumental in shaping the direction of this research and refining its outcomes.

I am also grateful to Vel Tech University for providing the necessary academic resources, computational facilities, and an excellent research environment that enabled the successful execution of this project. The support from faculty members and peers has been invaluable in enhancing my understanding of AI applications in healthcare.

Additionally, I would like to acknowledge the contributions of the broader research community, particularly the developers of open-source tools such as CodeCarbon, TensorFlow, and Scikit-learn, which facilitated the implementation and evaluation of AI models in this study. Their work has been a crucial foundation for this research.

Finally, I extend my heartfelt gratitude to my family and friends for their unwavering support, motivation, and patience throughout this journey. Their encouragement has been a source of strength in completing this research successfully. This work is a step toward making AI more effective in healthcare, and I hope it contributes meaningfully to the advancement of medical practices and patient safety.

VI. REFERENCES

REFERENCES

[1] Zhao, X., Wang, Y. (2025). Can Artificial Intelligence Technology Reduce Carbon Emissions? A Global Perspective. Energy Economics, 143, 108285.

[2] Zhang, Z., Leng, D., Li, Y., Gui, X., Cheng, Y., Zhao, J., Zhang, Z. (2024). Artificial Intelligence-Driven Insights: Precision Tracking of Power Plant Carbon Emissions Using Satellite Data. Energy Conversion and Economics, 5(5), 293-300.

[3] Akinlabi, E.T., Akinlabi, S. (2024). Monitoring Carbon Emissions Using Deep Learning and Statistical Process Control: A Strategy for Impact Assessment of Governments' Carbon Reduction Policies. Environmental Monitoring and Assessment, 196, Article 231.

[4] Zhong, J., Zhong, Y., Han, M., Yang, T., Zhang, Q. (2024). The Impact of AI on Carbon Emissions: Evidence from 66 Countries. Applied Economics, 56(25), 2975-2989.

[5] Luccioni, S., Dandres, T. (2023). Quantifying the Carbon Footprint of AI: A Case Study on BLOOM. Journal of Machine Learning Research, 24(1), 1-25.

[6] Wang, L., Chen, H. (2023). AI and Energy Consumption: Evaluating the Environmental Impact of AI Data Centers. Energy Policy, 165, 112927.

[7] Zhou, F., Li, X. (2023). AI in Energy Management: Reducing Carbon Footprint through Intelligent Systems. Renewable and Sustainable Energy Reviews, 152, 111678.

[8] Schwartz, R., Dodge, J., Smith, N.A., Etzioni, O. (2022). Green AI: Promoting Environmental Sustainability in AI Research. Communications of the ACM, 65(12), 27-29.

[9] Hao, K. (2022). Training a Single AI Model Can Emit as Much Carbon as Five Cars in Their Lifetimes. MIT Technology Review.

[10] Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., Dean, J. (2021). Carbon Emissions and Large Neural Network Training. arXiv preprint arXiv:2104.10350.

[11] Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., Pineau, J. (2020). Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning. Journal of Machine Learning Research, 21(1), 1-43.

[12] Lacoste, A., Luccioni, A., Schmidt, V., Dandres, T. (2019). Quantifying the Carbon Emissions of Machine Learning. arXiv preprint arXiv:1910.09700.

[13] Strubell, E., Ganesh, A., McCallum, A. (2019). Energy and Policy Considerations for Deep Learning in NLP. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 36453650.

[14] Dhar, P. (2018). The Carbon Impact of Artificial Intelligence. Nature Machine Intelligence, 1(1), 1-3.

[15] Nguyen, K.A., Sahin, O., Stewart, R.A., Zhang, H. (2017). Smart Technologies in Reducing Carbon Emission: Artificial Intelligence and Smart Water Meter. Proceedings of the 9th International Conference on Machine Learning and Computing, 517–522.

[16] Smith, J.A., Brown, L.B. (2017). Utilizing Machine Learning for Carbon Footprint Reduction in Manufacturing. Journal of Cleaner Production, 142, 915-925.

[17] Garcia, M., Lee, T. (2017). AI-Driven Energy Management Systems for Sustainable Buildings. Energy and Buildings, 150, 58-66.